

VS7 KNOWLEDGE SYSTEM

GIT

The History Behind Every Build

JULY 2026

Table of Contents

Part 0: Before Git

1. What Version Control Solves
 2. Installing Git & Configuration
 3. Terminal Basics You Need for Git
-

Part 1: Core Concepts

4. What Is a Repository
 5. Working Directory, Staging Area & Commits (The Three Trees)
 6. git init
 7. git status
 8. git add
 9. git rm
 10. git mv
 11. git commit
 12. .gitignore
 13. git log
 14. git show
-

Part 2: Branching & Merging

15. Branches as Pointers
16. git branch, git checkout & git switch
17. git restore
18. Merging Branches
19. Merge Conflicts
20. Fast-Forward vs 3-Way Merge

Part 3: Remote Collaboration

- 21. git remote
 - 22. git push, git pull & git fetch
 - 23. git clone
 - 24. Tracking Branches
 - 25. GitHub Basics: Pull Requests & Issues
-

Part 4: Rewriting History

- 26. Rebase vs Merge
 - 27. Interactive Rebase
 - 28. Cherry-Pick
 - 29. git reset vs git revert
 - 30. git stash
 - 31. git commit --amend
-

Part 5: Internals

- 32. The Object Model: Blobs, Trees & Commits
 - 33. The .git Directory Structure
 - 34. Refs
 - 35. The Reflog
 - 36. Hashing & Content Addressing
-

Part 6: Advanced & Team Workflows

- 37. git bisect
- 38. Git Hooks
- 39. Submodules
- 40. Worktrees
- 41. Tags
- 42. git blame
- 43. git clean
- 44. Branching Strategies: Git Flow vs Trunk-Based

Part 7: Common Mistakes & Recovery

45. Recovering Deleted Branches & Commits

46. Undoing a Bad Push

47. Force-Push Dangers

Appendix

48. Command Cheat Sheet

What Version Control Solves

Imagine you're writing a report in Word. You save it as `report.docx`. The next day you make big changes, but you're not sure they're good, so you save a copy as `report_final.docx` just in case. Two days later you tweak it again and save `report_final_v2.docx`. Then your teammate emails you their edits as `report_final_v2_johns_edits.docx`. By the end of the week you have eleven files in a folder and nobody — including you — can say which one is actually correct.

This is how most people manage files before they learn version control, and it breaks down fast:

- **No real history.** If you overwrite `report.docx` by saving on top of it, the previous version is just gone. There is no "undo" once you've closed the file and reopened your computer the next day.
- **Collaboration by email or shared drive.** Two people editing the same file at the same time either overwrite each other's work, or you end up manually copy-pasting each other's paragraphs to merge changes.
- **No way to explain a change.** Even if you kept every version, nothing tells you *why* a change was made, or which lines actually differ between `v2` and `v3`.
- **Fear of experimenting.** Without a safety net, you're afraid to try a risky change to your code or document, because if it goes wrong you might not be able to get back to where you started.

BEGINNER PRIMER

WHAT IT IS

Version control is a system that automatically records every change you make to a set of files, so you can see, compare, and return to any past version at any time.

WHY IT EXISTS

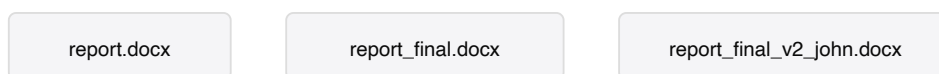
It replaces the "save a new copy with a new filename" habit with one real file whose entire history is tracked automatically. It also gives multiple people a reliable way to combine their changes to the same files without emailing zip files back and forth.

What Actually Happens

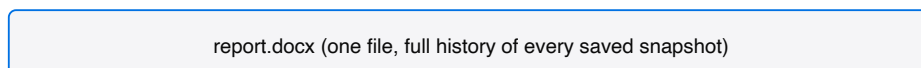
A version control tool sits quietly alongside your project folder. Every time you tell it "save this as a checkpoint," it records a snapshot of every file at that exact moment, along with who made the change, when, and a short message describing why.

Those snapshots build up into a timeline. You can look back at any snapshot, compare two snapshots to see exactly what changed line by line, or jump back to an older snapshot if the current state is broken — without losing anything in between.

Without version control



With version control



Git, Specifically

Git is the version control tool this entire course teaches. It was created in 2005 by Linus Torvalds (the creator of Linux) to manage the Linux kernel's source code, which involves thousands of contributors changing the same files.

Git works on any files, not just code — but it's most widely used for source code because text files (like `.py`, `.js`, or `.html`) let Git show precisely which lines changed. Later sections in this course will install it and start using it for real.

COMMON MISTAKE / GOTCHA

- **Mistake:** Thinking version control is only for programmers. Anyone editing text-based files over time — writers, researchers, designers working with config files — hits the exact same "which version is correct" problem. Git is most common in software, but the underlying problem it solves is universal.

QUICK CHECK

- Name two concrete problems with managing file versions by renaming files like `file_final_v2.docx`.
- What does version control record every time you make a checkpoint/snapshot?

Installing Git & Configuration

BEGINNER PRIMER

WHAT IT IS

Git is a program you install on your computer, and configuration is a small one-time setup step that tells Git who you are.

WHY IT EXISTS

Git needs to be installed before any `git` command works at all. It also needs your name and email so that every snapshot it records can say who made it — without that, teammates couldn't tell whose change is whose.

Installing Git

The install method depends on your operating system.

CODE

```
# macOS – via Homebrew (recommended if you have it)
$ brew install git

# macOS – alternative: install Xcode Command Line Tools
$ xcode-select --install

# Ubuntu / Debian Linux
$ sudo apt update && sudo apt install git

# Fedora / RHEL Linux
$ sudo dnf install git

# Windows – download and run the installer from
# https://git-scm.com/download/win (installs "Git for Windows",
# which includes Git Bash – a terminal to run these commands)
```

Confirm it installed correctly by checking the version:

CODE

```
$ git --version
git version 2.43.0
```

The Command(s)

Set your identity once after installing. Git attaches this name and email to every commit you make.

CODE

```
$ git config --global user.name "Varun Kumar"
$ git config --global user.email "varunkumar.u7@gmail.com"

# verify what was set
```

```
$ git config --list
user.name=Varun Kumar
user.email=varunkumar.u7@gmail.com
core.editor=vim
init.defaultbranch=main
```

What Actually Happens

The `--global` flag writes these settings to a single config file for your user account (`~/.gitconfig` on Mac/Linux), so they apply to every repository you work in on that machine.

Drop `--global` and run the same command *inside* a specific project folder to set a value for just that one repository — useful if you use a different email for work versus personal projects. A per-repo setting overrides the global one for that project only.

COMMON MISTAKE / GOTCHA

- **Mistake:** Skipping `user.name` / `user.email` setup and only discovering it's missing when a commit fails or gets recorded with a placeholder identity like `you@example.com`. Set both before your very first commit — fixing authorship on old commits later is far more painful than typing two commands now.

QUICK CHECK

- What is the difference between running `git config --global` and `git config` without `--global`?
- Which command shows every config value currently set?

Terminal Basics You Need for Git

BEGINNER PRIMER

WHAT IT IS

The terminal is a text-based window where you type commands instead of clicking icons, and it's where every `git` command in this course gets typed.

WHY IT EXISTS

Git has no required visual interface — its native form is command-line. Before typing a single `git` command, you need to know how to move between folders and create files from the terminal, since every example from here on starts "open your terminal and...".

This section is not a full shell course — just the five commands you need to follow along.

The Command(s)

CODE

```
# pwd – "print working directory": shows where you currently are
$ pwd
/Users/varun/projects

# ls – lists the files and folders in the current directory
$ ls
notes.txt  photos  old-report.docx

# cd – "change directory": move into a folder
$ cd photos
$ pwd
/Users/varun/projects/photos

# cd .. – move UP one level, back to the parent folder
$ cd ..
```

```
$ pwd
/Users/varun/projects

# mkdir - "make directory": create a new folder
$ mkdir my-first-repo

# touch - create a new, empty file
$ cd my-first-repo
$ touch readme.txt
$ ls
readme.txt
```

What Actually Happens

Your terminal always has a "current location" in your computer's folder structure — that's what `pwd` reports. Every command you run (including every `git` command) acts on the current location unless you tell it otherwise.

`cd` changes that location, `ls` shows what's inside it, and `mkdir` / `touch` create new folders and files inside it. Once you can combine these to land inside a project folder, you're ready for `git init` in a later section.

Command	What it does
<code>pwd</code>	Print the full path of the folder you're currently in
<code>ls</code>	List files and folders in the current folder
<code>cd <folder></code>	Move into <code><folder></code>
<code>cd ..</code>	Move up one level, to the parent folder
<code>mkdir <name></code>	Create a new, empty folder called <code><name></code>
<code>touch <name></code>	Create a new, empty file called <code><name></code>

COMMON MISTAKE / GOTCHA

- **Mistake:** Running `git` commands from the wrong folder. Git only sees the folder you're currently "in" (and its subfolders) — if you meant to work on a project in `~/projects/my-app` but never `cd` 'd into it, Git commands will silently act on the wrong place or fail with "not a git repository." Always confirm with `pwd` when unsure.

QUICK CHECK

- Which command tells you exactly which folder you're currently in?
- What is the difference between `mkdir` and `touch`?

What Is a Repository

BEGINNER PRIMER

WHAT IT IS

A repository (or "repo") is a project folder that Git is watching, made of your regular files plus one hidden folder that stores their entire history.

WHY IT EXISTS

A plain folder only ever holds the current state of your files — save over a file and the old version is gone. A repository keeps every recorded snapshot of that folder over time, which is what makes "undo to yesterday's version" possible.

Regular Folder vs. Repository

Picture a normal project folder on your computer: it has your files in it, nothing more. Now picture that same folder after running `git init` (covered in the next section) — the files look identical, but a new hidden folder named `.git` has appeared alongside them.

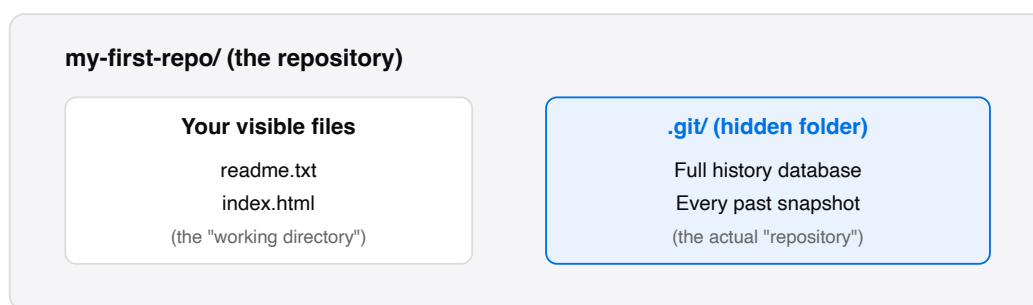
CODE

```
$ ls -a my-first-repo  
.  
..  
.git  
readme.txt
```

What Actually Happens

That `.git` folder is the repository. It's not just a settings file — it's a self-contained database that stores every snapshot you've ever recorded, who made each one, when, and why.

Your visible files (`readme.txt` and friends) are called the **working directory** — the files you can see and edit in Finder/Explorer or your editor. The `.git` folder is separate from them and is what actually gives the folder "memory."



Delete the `.git` folder and the history is gone forever — the folder becomes a plain, untracked directory again, even though the visible files are untouched. This is why `.git` is the single most important folder in any Git project.

COMMON MISTAKE / GOTCHA

- **Mistake:** Assuming a folder is "a Git repo" just because it contains code or looks like a typical project. A folder is only a repository if it has a `.git` subfolder inside it. Many beginners write code for weeks in a plain folder, assuming Git is somehow tracking it automatically — it isn't, until you run `git init`.

QUICK CHECK

- What actually distinguishes a Git repository from a regular folder?
- What happens to a folder's history if you delete its `.git` subfolder?

Working Directory, Staging Area & Commits (The Three Trees)

BEGINNER PRIMER

WHAT IT IS

Every change you make in Git passes through three stages before it's permanently recorded: the working directory, the staging area, and the commit history.

WHY IT EXISTS

Without a middle stage, Git would have to save every single file change immediately, with no way to choose which changes belong together. The staging area lets you build up exactly the set of changes you want in one snapshot before recording it — like assembling ingredients before you commit to cooking the dish.

The Three Trees

Git calls these three areas "trees" internally. You don't need that term day-to-day — what matters is what each one holds and how a change moves between them.

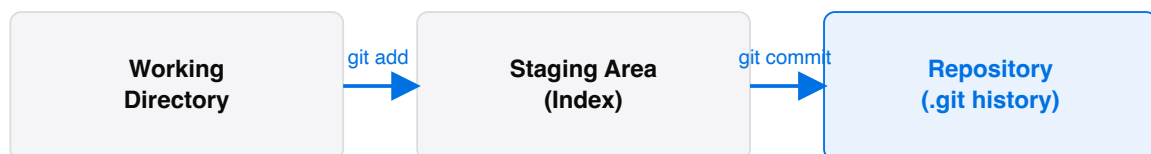
Area	What it holds	You move things here with
Working Directory	The actual files you see and edit on disk right now	Just editing/saving a file normally

Staging Area (Index)	A holding area listing exactly which changes will go into the <i>next</i> commit	<code>git add</code>
Repository (.git history)	Permanent, named snapshots — the actual project history	<code>git commit</code>

What Actually Happens

You edit a file — it now differs from the last snapshot, and Git considers it "modified" but not yet tracked for the next commit. Running `git add <file>` copies that change into the staging area, marking it "ready to be committed." Running `git commit` takes everything currently in the staging area and permanently records it as a new snapshot in the repository's history.

This is the single most important mental model in Git — nearly every command you'll learn moves a change between these three areas, or inspects which area a change currently sits in.



The Command(s)

CODE

```
# edit a file in the working directory (just save it normally)
$ echo "hello world" > notes.txt

# stage the change – moves it into the staging area
$ git add notes.txt

# commit – permanently records everything in the staging area
$ git commit -m "Add notes.txt with a greeting"
```

```
[main a1b2c3d] Add notes.txt with a greeting
1 file changed, 1 insertion(+)
create mode 100644 notes.txt
```

COMMON MISTAKE / GOTCHA

- **Mistake:** Editing a file and running `git commit` directly, expecting the edit to be included. Unstaged changes are NOT included in a commit — only what's in the staging area gets committed. Skipping `git add` is the most common reason a beginner's commit doesn't contain the change they just made.

QUICK CHECK

- Name the three areas a change passes through, in order, from editing a file to it being permanently recorded.
- If you edit a file but never run `git add`, will `git commit` include that change? Why or why not?

git init

BEGINNER PRIMER

WHAT IT IS

`git init` turns a regular folder into a Git repository by creating the hidden `.git` folder inside it.

WHY IT EXISTS

Git doesn't track every folder on your computer automatically — that would be slow and pointless for folders that aren't projects. You have to explicitly opt a folder in, once, and `git init` is that command.

The Command(s)

CODE

```
$ mkdir my-first-repo
$ cd my-first-repo
$ git init
Initialized empty Git repository in /Users/varun/projects/my-first-repo/.git/

$ ls -a
.    ..    .git
```

You can also initialize a repository in an existing folder that already has files in it — nothing gets deleted or changed, Git just starts watching it.

CODE

```
$ cd existing-project
$ ls
app.py    requirements.txt

$ git init
Initialized empty Git repository in /Users/varun/projects/existing-project/.git/

$ ls
app.py    requirements.txt    .git
```

What Actually Happens

Right after `git init`, the folder is a repository, but it's an *empty* one — no commits exist yet, and Git isn't tracking your existing files automatically. It has simply created the `.git` database and is now watching the folder for changes you explicitly tell it about (via `git add`, covered in section 5).

Running `git init` again in a folder that's already a repository is safe — it does nothing destructive, it just notices `.git` already exists and leaves it alone.

COMMON MISTAKE / GOTCHA

- **Mistake:** Running `git init` in your home folder or a very high-level parent folder (like `~/Documents`) by accident. This makes Git think your *entire* Documents folder is one giant repository, which causes confusing behavior later. Always check `pwd` before running `git init` to confirm you're inside the specific project folder you meant to initialize.

QUICK CHECK

- What does `git init` actually create on disk?
- After running `git init` in a folder with existing files, are those files automatically tracked by Git?

git status

BEGINNER PRIMER

WHAT IT IS

`git status` tells you, in plain English, exactly which files have changed and which of the three trees (from section 5) each change currently sits in.

WHY IT EXISTS

Without it, you'd have no way to know whether your edits are still sitting only in the working directory, staged and ready to commit, or already safely committed. It's the command you'll run more than any other — a sanity check before almost every `git add` or `git commit`.

The Command(s)

CODE

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    notes.txt

nothing added to commit but untracked files present (use "git add" to track)
```

After staging the file, the same command shows a different picture:

CODE

```
$ git add notes.txt
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   notes.txt
```

After committing, and then editing the file again:

CODE

```
$ git commit -m "Add notes.txt"
$ echo "one more line" >> notes.txt
$ git status
On branch main

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
```

```
(use "git restore <file>..." to discard changes in working directory)
```

```
modified:   notes.txt
```

```
no changes added to commit (use "git add" and/or "git commit -a")
```

What Actually Happens

Each status message maps directly onto the three-trees model from section 5. Git is telling you, per file, which of the three areas its latest change lives in right now.

Status label	Meaning	Tree it's in
Untracked	Git has never seen this file before — it isn't part of any commit or the staging area	Working directory only
Changes not staged for commit / "modified"	A previously committed file was edited, but the edit hasn't been staged	Working directory only
Changes to be committed / "new file" or "modified"	The change is staged and will be included in the next commit	Staging area
(file not listed at all)	Working directory matches the last commit exactly — nothing to report	Repository (already committed)

COMMON MISTAKE / GOTCHA

- **Mistake:** Seeing a file listed under both "Changes to be committed" and "Changes not staged for commit" at the same time, and assuming `git status` is broken. This happens when you stage a file, then edit it again afterward — the staged version and the newer working-directory version are genuinely different, so Git reports both truthfully. Re-run `git add` to stage the newest edit too.

QUICK CHECK

- A file shows as "Untracked" in `git status`. What tree is it in, and what command moves it toward being committed?
- Why might the same file appear in both the staged and not-staged sections of `git status` output at once?

git add

BEGINNER PRIMER

WHAT IT IS

`git add` moves changes from the working directory into the staging area, marking them to be included in the next commit.

WHY IT EXISTS

You rarely want every edit in your working directory bundled into one commit. Staging lets you pick exactly which changes go into the next snapshot — even individual lines within a file — so commits stay focused and reviewable.

The Command(s)

CODE

```
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
       modified:   index.html
       modified:   style.css

Untracked files:
  (use "git add <file>..." to include in what will be committed)
       notes.txt
```

```
$ git add index.html          # stage one specific file
$ git add style.css notes.txt # stage several files at once
$ git add .                  # stage everything in and below the current
                             # directory
$ git add -A                 # stage everything in the whole repo,
                             # including deletions elsewhere
```

What Actually Happens

Git reads the current content of the file from the working directory and writes it into the staging area (the index). The file itself does not move or change — only a copy of its current state is recorded as "ready to commit."

If you edit the file again after staging, the working directory copy changes but the staged copy does not. `git status` will then show the same file as both staged and modified — two different versions in play at once.

Staging Part of a File: `git add -p`

`git add -p` (patch mode) walks through a file's changes in chunks called "hunks" and asks whether to stage each one.

CODE

```
$ git add -p index.html
diff --git a/index.html b/index.html
index e69de29..8f3a1c2 100644
--- a/index.html
+++ b/index.html
@@ -12,6 +12,7 @@
     <body>
+    <h1>Welcome</h1>
     </body>
 </html>
Stage this hunk [y,n,q,a,d,s,e,?]? y
```

This is how experienced developers split one messy work session into several clean, single-purpose commits.

COMMON MISTAKE / GOTCHA

- **Mistake:** Running `git add .` without checking `git status` first, and accidentally staging a debug file, a build artifact, or a stray `.env` you never meant to commit. Always glance at `git status` (or `git diff --staged`) before running `git commit`.

QUICK CHECK

- If you stage a file with `git add` and then edit it again, what does `git status` show?
- What's the difference between `git add .` and `git add -A`?

git rm

BEGINNER PRIMER

WHAT IT IS

`git rm` deletes a tracked file from the working directory and stages that deletion in one step.

WHY IT EXISTS

Deleting a file with Finder or the plain `rm` command only touches your working directory — Git still thinks the file should exist, because the staging area and history haven't been told about the deletion. `git rm` keeps all three trees in sync in a single command.

The Command(s)

CODE

```
$ git rm old-report.pdf
rm 'old-report.pdf'

$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    old-report.pdf
```

What Actually Happens

`git rm <file>` does two things at once: it removes the file from your working directory (like `rm` would), and it stages that removal so the next commit records the file as deleted.

Compare that to just deleting the file yourself outside Git:

CODE

```
$ rm old-report.pdf
$ git status
On branch main
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)
        deleted:    old-report.pdf
```

Git notices the file is gone from disk, but the deletion is **unstaged** — you'd still need to run `git add old-report.pdf` (yes, `add` stages deletions too) or `git rm old-report.pdf` to record it for the next commit.

Untracking Without Deleting: `--cached`

Sometimes you want Git to stop tracking a file but keep it on disk — for example, a config file you accidentally committed.

CODE

```
$ git rm --cached secrets.local.json
rm 'secrets.local.json'

$ ls secrets.local.json
secrets.local.json  # still there in the working directory

$ git status
On branch main
Changes to be committed:
  deleted:    secrets.local.json
Untracked files:
  secrets.local.json
```

The file becomes untracked going forward, but it isn't erased. Pair this with adding the file to `.gitignore` so it doesn't get re-added by accident.

COMMON MISTAKE / GOTCHA

- **Mistake:** Running `rm` (the shell command, not Git's) on a tracked file and assuming the deletion is committed. It's only staged after you run `git add` or `git rm` on it afterward — until then, `git status` shows "deleted, not staged for commit," and the deletion is not part of history yet.

QUICK CHECK

- If you delete a tracked file with Finder instead of `git rm`, what does `git status` show, and what's missing?
- What does `git rm --cached` do differently from plain `git rm`?

git mv

BEGINNER PRIMER

WHAT IT IS

`git mv` renames or moves a tracked file and stages that change, in a single command.

WHY IT EXISTS

Renaming a file with the plain `mv` command leaves Git seeing two separate changes: an untracked new file and a deleted old one. `git mv` is a shortcut that does the rename and the staging for you, so you don't have to run three commands to do one thing.

The Command(s)

CODE

```
$ git mv index.html home.html

$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        renamed:    index.html -> home.html
```

What Actually Happens

`git mv old new` is a convenience wrapper. It's exactly equivalent to running these three commands yourself:

CODE

```
$ mv old new
$ git rm old
$ git add new
```

Git doesn't actually store a "this file was renamed" record in a commit. It stores file snapshots. When you run `git log` or `git diff` later, Git compares the old and new snapshots and, if their content is similar enough, detects and displays the change as a rename automatically — even if you'd done the rename with plain `mv` and separate `add` / `rm` calls instead of `git mv`.

COMMON MISTAKE / GOTCHA

- **Mistake:** Assuming `git mv` is required to get a "renamed" label in your history. It isn't — rename detection is based on content similarity at diff/log time, not on which command you used. If you heavily edit a file in the same commit as renaming it, Git may fail to detect the rename and show it as a plain delete + add instead.

QUICK CHECK

- What three separate commands does `git mv` replace?
- Does Git store renames as a distinct type of history entry, or detect them after the fact?

git commit

BEGINNER PRIMER

WHAT IT IS

`git commit` takes everything currently in the staging area and saves it as a permanent, named snapshot in the project's history.

WHY IT EXISTS

Staging alone doesn't create history — it only prepares what the next snapshot will contain. Committing is the moment that snapshot becomes real, gets a unique hash, and becomes something you can return to, compare against, or share.

The Command(s)

CODE

```
$ git commit -m "Add hero section to landing page"
[main a1b2c3d] Add hero section to landing page
 2 files changed, 18 insertions(+), 1 deletion(-)
```

Without `-m`, Git opens your configured text editor so you can write a longer message:

CODE

```
$ git commit
# opens editor with:
#
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch main
# Changes to be committed:
#       modified:   index.html
```

What Actually Happens

Git bundles the staged content into a new commit object, records the author, timestamp, commit message, and a pointer to the previous commit (its parent), then moves the current branch pointer forward to this new commit.

Only what was staged gets committed. Anything left unstaged in the working directory stays out of this snapshot entirely — that's the whole point of the staging area.

Skipping the Stage: `git commit -a`

CODE

```
$ git commit -am "Fix typo in footer"
[main d4e5f6a] Fix typo in footer
1 file changed, 1 insertion(+), 1 deletion(-)
```

`-a` automatically stages every file Git already tracks that has been modified or deleted, then commits. It's a shortcut for `git add -u` followed by `git commit`.

COMMON MISTAKE / GOTCHA

- **Mistake:** Assuming `-a` stages everything, including new files. It only picks up changes to files Git is *already tracking*. A brand-new untracked file (say, a new component you just created) is silently left out of the commit — you still need an explicit `git add` for it first.

Commit Message Conventions

- Summary line: short (roughly 50 characters), imperative mood — "Add login form," not "Added login form" or "Adds login form."
- Treat the summary line like the answer to: "If applied, this commit will ____."
- Leave a blank line, then optional body text explaining *why*, for anything non-obvious.

QUICK CHECK

- If you create a new file, edit an existing tracked file, and run `git commit -am "..."`, which of the two changes ends up in the commit?
- What information does a commit object store besides the file snapshot itself?

.gitignore

BEGINNER PRIMER

WHAT IT IS

`.gitignore` is a plain text file that tells Git which files or folders to never show as untracked and never stage automatically.

WHY IT EXISTS

Every project generates files Git shouldn't track — build output, dependency folders, log files, local secrets. Without a way to hide them, `git status` would be cluttered with noise and someone could easily commit a giant folder or an API key by mistake.

The Command(s)

CODE

```
$ cat .gitignore
*.log
node_modules/
.env
dist/
.DS_Store

$ git status
On branch main
nothing to commit, working tree clean
```

Even though `debug.log`, `node_modules/`, and `.env` exist on disk, Git no longer mentions them.

What Actually Happens

Each line in `.gitignore` is a pattern. Git checks every untracked file in the working directory against these patterns; a match means the file is hidden from `git status`, from tab-completion, and from `git add .`

- `*.log` — any file ending in `.log`, anywhere in the repo
- `node_modules/` — the whole folder and everything inside it (trailing slash means "directory")
- `.env` — an exact filename match

Per-Repo vs Global

Scope	File	Use for
Per-repo	<code>.gitignore</code> in the project root, committed to the repo	Rules everyone on the project needs (build output, dependencies)
Global	<code>~/.gitignore_global</code> , set via <code>git config --global core.excludesfile ~/.gitignore_global</code>	Personal editor/OS files (<code>.DS_Store</code> , <code>*.swp</code>) that shouldn't clutter every project's shared <code>.gitignore</code>

COMMON MISTAKE / GOTCHA

- **Mistake:** Adding a file to `.gitignore` after it's already been committed, and expecting Git to stop tracking it. `.gitignore` only affects *untracked* files. A file Git already knows about keeps being tracked and shown in diffs until you explicitly remove it with `git rm --cached <file>`.

QUICK CHECK

- You add `secrets.json` to `.gitignore`, but `git status` still shows it as modified. Why, and how do you fix it?
- Where would you put a rule to ignore `.DS_Store` across every repo on your machine, not just one project?

git log

BEGINNER PRIMER

WHAT IT IS

`git log` prints the commit history of your repository, newest commit first.

WHY IT EXISTS

Commits are the whole point of using Git, but they're invisible unless you can look back at them. `git log` is how you answer "what happened, when, and who did it" — essential for tracking down when a bug was introduced or reviewing recent work.

The Command(s)

CODE

```
$ git log
commit a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0 (HEAD -> main)
Author: Varun Kumar <varun@example.com>
Date:   Sun Jul 19 10:42:03 2026 +0530

    Add hero section to landing page

commit 9f8e7d6c5b4a3f2e1d0c9b8a7f6e5d4c3b2a1f0e
Author: Varun Kumar <varun@example.com>
Date:   Sat Jul 18 16:05:11 2026 +0530
```

Fix typo in footer

commit 4c3b2a1f0e9d8c7b6a5f4e3d2c1b0a9f8e7d6c5b

Author: Varun Kumar <varun@example.com>

Date: Sat Jul 18 09:20:44 2026 +0530

Initial commit

Condensed Views

CODE

```
$ git log --oneline
```

```
a1b2c3d Add hero section to landing page
```

```
9f8e7d6 Fix typo in footer
```

```
4c3b2a1 Initial commit
```

```
$ git log --oneline --graph --all
```

```
* a1b2c3d (HEAD -> main) Add hero section to landing page
```

```
* 9f8e7d6 Fix typo in footer
```

```
* 4c3b2a1 Initial commit
```

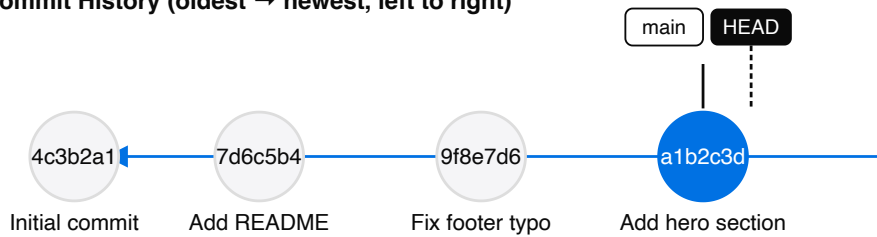
`--oneline` shows a shortened hash plus the summary line. `--graph` draws the branch/merge structure with `*` and `|` characters. `--all` includes every branch, not just the one you're currently on.

What Actually Happens

Every commit stores a pointer to its parent commit — the one it was created on top of. `git log` starts at `HEAD` (your current position) and walks backward through these parent pointers, printing each commit it finds.

A branch name like `main` is just a movable label pointing at one commit. `HEAD` is a label pointing at whichever commit (usually via a branch) you currently have checked out. This chain of parent pointers is often called the commit graph or DAG (directed acyclic graph).

Commit History (oldest → newest, left to right)



Each commit points backward to its parent — that's why `git log` reads newest-first: it starts at `HEAD` / `main` on the right and follows the arrows left, back through history.

COMMON MISTAKE / GOTCHA

- **Mistake:** Running plain `git log` in a terminal and getting stuck in the pager (it opens in `less` by default). Press `q` to exit. Reaching for `--oneline` in daily use avoids this almost entirely since short output usually fits on screen.

QUICK CHECK

- What does each commit store that lets `git log` walk backward through history?
- What's the difference between what `main` points to and what `HEAD` points to?

git show

BEGINNER PRIMER

WHAT IT IS

`git show` displays the full details of a single commit: its metadata and the exact changes it introduced.

WHY IT EXISTS

Sometimes you don't need the whole history, just "what did this one commit actually change?" — for example, when reviewing a specific commit hash mentioned in a bug report. `git show` answers that directly without scrolling through unrelated commits.

The Command(s)

CODE

```
$ git show a1b2c3d
commit a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0 (HEAD -> main)
Author: Varun Kumar <varun@example.com>
Date:   Sun Jul 19 10:42:03 2026 +0530

    Add hero section to landing page

diff --git a/index.html b/index.html
index e69de29..8f3a1c2 100644
--- a/index.html
+++ b/index.html
@@ -10,6 +10,9 @@
     <body>
+    <section class="hero">
+        <h1>Welcome</h1>
+    </section>
     </body>
 </html>

$ git show      # with no argument, shows HEAD (the current commit)
```

What Actually Happens

`git show <commit>` prints the commit's metadata (author, date, message) exactly like a single entry in `git log`, then appends the diff between that commit and its parent — the actual lines added and removed.

You can also point it at a specific file within a commit to see only that file's change:

```
git show a1b2c3d -- index.html
```

git show vs git log -p

Command	Scope
<code>git show <commit></code>	One commit's diff and metadata
<code>git log -p</code>	Every commit in the log's diff, one after another — the whole range

CODE

```
$ git log -p
commit a1b2c3d... (HEAD -> main)
    Add hero section to landing page
diff --git a/index.html b/index.html
...

commit 9f8e7d6...
    Fix typo in footer
diff --git a/footer.html b/footer.html
...
```

`git log -p` is `git show` repeated for every commit it lists — useful for reviewing a whole stretch of history, but noisy if you only care about one commit.

COMMON MISTAKE / GOTCHA

- **Mistake:** Running `git log -p` on a large repo and getting overwhelmed by hundreds of diffs printed one after another. When you already know which commit you want, `git show <hash>` is almost always the faster, more targeted tool.

QUICK CHECK

- What two things does `git show` print for a given commit?
- If you wanted diffs for every commit made in the last week, would you reach for `git show` or `git log -p`?

Branches as Pointers

BEGINNER PRIMER

WHAT IT IS

A branch is just a movable label pointing at one commit — nothing more.

WHY IT EXISTS

Without branches, trying an idea means risking your only line of history. A branch gives you a cheap, disposable pointer you can create, move, and delete without touching a single commit object.

The Command(s)

CODE

```
$ git branch                # list branches, * marks current
* main

$ cat .git/refs/heads/main  # a branch really is just a file
a1b2c3d4e5f6...           # containing one commit hash

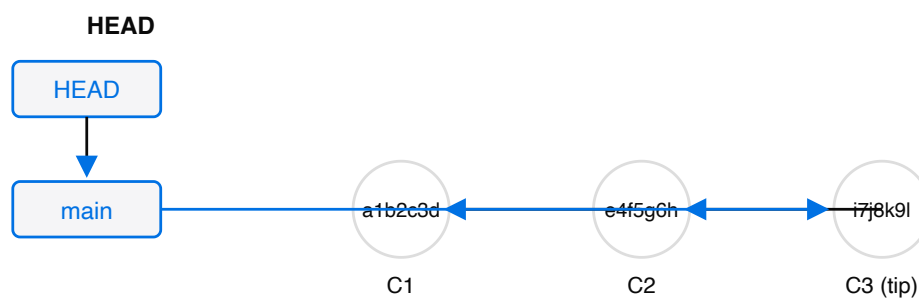
$ cat .git/HEAD             # HEAD is a file pointing at a branch
ref: refs/heads/main
```

What Actually Happens

Every commit already points backward to its parent — that's the DAG from Part 1. A branch adds one more pointer on top: a 41-byte text file under `.git/refs/heads/` holding the hash of a single commit.

Committing on a branch does two things: it creates a new commit object pointing at the old tip as its parent, then it rewrites the branch file to hold the new commit's hash. The branch itself never "contains" commits — it just points at the latest one, and you walk backward from there.

HEAD is a separate pointer, and normally it doesn't point at a commit directly — it points at a branch, which points at the commit. This indirection is why `git commit` automatically advances whichever branch you're on: Git resolves HEAD → branch → commit, and only the branch file gets rewritten.



COMMON MISTAKE / GOTCHA

- **Mistake:** Thinking a branch is a container that "holds" its commits, like a folder holds files. It isn't — it's a single hash. Delete the branch and the commits usually still exist (reachable from other refs, or dangling) until garbage collection actually removes them.

QUICK CHECK

- If a branch is just a pointer to one commit, how does Git know the full history behind that branch?
- Why does HEAD point at a branch instead of directly at a commit in normal use?

git branch, git checkout & git switch

BEGINNER PRIMER

WHAT IT IS

`git branch` manages branch pointers; `git switch` moves HEAD between them.

WHY IT EXISTS

You need a way to create parallel lines of work and jump between them without losing anything. Branch pointers make that jump instant — no files get copied, only which commit HEAD resolves to changes.

Instructor Note: Why git checkout Is Confusing

`git checkout` is one command doing two unrelated jobs, and the syntax alone doesn't tell you which one is running.

CODE

```
$ git checkout main          # job 1: switch branches — HEAD now points at
main
Switched to branch 'main'

$ git checkout -- file.txt    # job 2: restore a file — discards uncommitted
edits

$ git checkout file.txt      # (the -- is often optional, same effect)
```

Both are spelled `git checkout <name>`. If `<name>` matches a branch, Git moves HEAD. If it matches a file path, Git overwrites that file from the index, silently discarding your edits. Type a branch name that happens to also be a filename in your working directory and you can get the wrong job.

This is historical baggage, not good design — `checkout` grew both meanings over the years because they both involve "replace what's here with what's recorded." Git 2.23 split them apart on purpose.

The Split: git switch and git restore

Old (still works)	New (recommended)	Job
<code>git checkout <branch></code>	<code>git switch <branch></code>	Move HEAD to a branch
<code>git checkout -- <file></code>	<code>git restore <file></code>	Discard file changes

CODE

```
$ git switch main          # unambiguous: only switches branches
Switched to branch 'main'

$ git switch -c feature/login # -c = create and switch in one step
Switched to a new branch 'feature/login'
```

`git switch` only ever touches branches. `git restore` only ever touches files. Neither can accidentally do the other's job — that's the entire point of the split. `git checkout` still exists and still works exactly as before for backward compatibility, but new workflows should reach for `switch` and `restore`.

git branch: Create, List, Delete

CODE

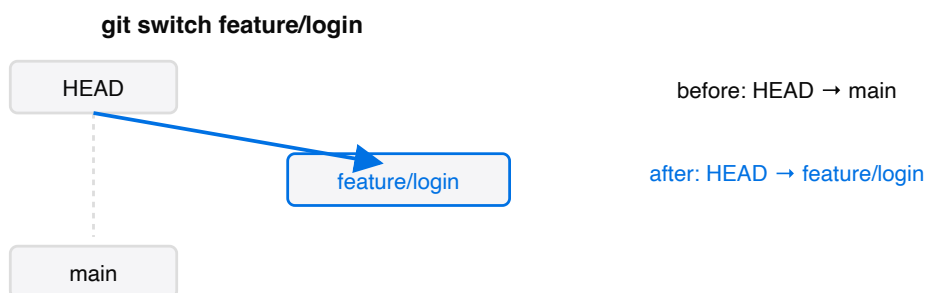
```
$ git branch feature/login    # create, does NOT switch to it
$ git branch                  # list local branches
    feature/login
* main
$ git branch -v               # list with last commit on each
    feature/login a1b2c3d Add login form
* main              e4f5g6h Update README

$ git branch -d feature/login # delete, safe: refuses if unmerged
error: The branch 'feature/login' is not fully merged.
$ git branch -D feature/login # force-delete, discards unmerged commits
```

What Actually Happens

`git switch <branch>` rewrites `.git/HEAD` to point at the new branch's ref, then updates your working directory and index to match that branch's tip commit. Nothing is copied or recomputed from scratch — Git just diffs the two commit trees and adjusts the files that differ.

`git branch <name>` just writes a new file under `.git/refs/heads/` pointing at HEAD's current commit — it does not move HEAD. You stay on your current branch until you explicitly switch.



COMMON MISTAKE / GOTCHA

- **Mistake:** Running `git checkout <name>` without checking whether `<name>` is a branch or a file. If you have both a branch called `report` and a file called `report`, Git picks the branch — but the ambiguity has burned real teams. Prefer `git switch` so there's nothing to guess.

QUICK CHECK

- Why did Git introduce two new commands instead of just fixing checkout in place?
- What's the difference between `git branch -d` and `git branch -D`?
- Does `git branch feature/login` move HEAD? Does `git switch -c feature/login`?

git restore

BEGINNER PRIMER

WHAT IT IS

`git restore` puts a file back to a previous state — working directory, staging area, or both.

WHY IT EXISTS

The previous section covered how `git checkout` used to handle file restoration as a side job bolted onto branch switching. `git restore` is the dedicated command for that job — file-only, nothing to do with branches or HEAD movement.

The Command(s)

CODE

```
$ git status
On branch main
Changes not staged for commit:
  modified:   config.py

$ git restore config.py          # discard working-dir changes, back to
last commit
$ git status
On branch main
nothing to commit, working tree clean
```

CODE

```
$ git add notes.txt
$ git status
Changes to be committed:
  modified:   notes.txt

$ git restore --staged notes.txt # unstage, but KEEP the edit
```

```
$ git status
Changes not staged for commit:
  modified:   notes.txt
```

What Actually Happens

`git restore <file>` overwrites the working-directory copy of that file with the version from the index (staged copy, or last commit if nothing's staged). Your edit is gone — there is no undo for this one.

`git restore --staged <file>` is a different, narrower operation: it copies the file from the last commit back into the index only, leaving your working-directory edit untouched. This is how you "unstage" a file after an accidental `git add` without losing the work.

Both are file-scoped and never touch HEAD or which branch you're on — that separation from `git switch` is exactly the point of the checkout split.

COMMON MISTAKE / GOTCHA

- **Mistake:** Running plain `git restore <file>` when you meant to unstage. Without `--staged`, it discards the working-directory edit permanently — it does not just move the file out of staging. If the file is staged AND you want to keep the edit but unstage it, you need `--staged`, not the bare command.

QUICK CHECK

- You staged a file with `git add`, then realize you want to keep editing before committing. Which restore command unstages it without losing your changes?
- What's the difference between what `git restore file.txt` and `git restore --staged file.txt` each overwrite?

Merging Branches

BEGINNER PRIMER

WHAT IT IS

Merging brings the changes from one branch into another, combining their histories.

WHY IT EXISTS

Branches let you work in isolation, but isolated work is useless if it never rejoins the main line. Merging is how separate lines of commits become one again, without losing either side's history.

The Command(s)

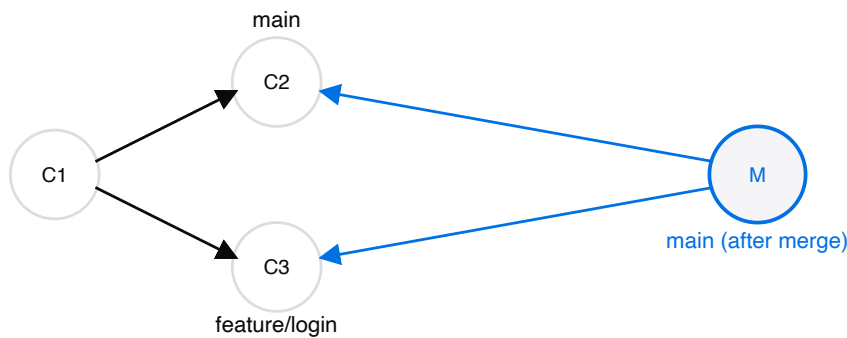
CODE

```
$ git switch main                # be ON the branch that should receive
changes
$ git merge feature/login        # bring feature/login's commits into
main
Merge made by the 'recursive' strategy.
 login.py | 12 ++++++++
1 file changed, 12 insertions(+)
```

What Actually Happens

You run `git merge <branch>` from the **receiving** branch — the one you're currently on. Git finds the common ancestor of the two branches, then combines the changes each side made since that ancestor.

If both branches added new commits since diverging, Git creates a new **merge commit** with two parents: the previous tip of `main` and the tip of `feature/login`. History now shows both lines joining at one point.



The merge commit's tree snapshot reflects the combined result of both branches' changes. If the same lines of the same file were touched on both sides, Git can't auto-combine them — that produces a conflict, covered next.

COMMON MISTAKE / GOTCHA

- **Mistake:** Running `git merge main` while standing on `feature/login`, meaning to bring `feature/login` into `main`. Merge direction is "merge THAT branch into the one I'm currently on" — always check `git branch` or your prompt for the current branch before merging.

QUICK CHECK

- If you want `feature/login`'s work inside `main`, which branch do you switch to before running `git merge`?
- How many parent commits does a merge commit have when both branches added new work?

Merge Conflicts

BEGINNER PRIMER

WHAT IT IS

A conflict is Git pausing a merge because two branches edited the same lines differently and it won't guess which one you want.

WHY IT EXISTS

Git can auto-combine changes to different parts of a file, but it has no way to know which of two competing edits to the same line is "correct." Rather than silently picking one, it stops and hands the decision to you.

The Command(s)

CODE

```
$ git merge feature/login
Auto-merging config.py
CONFLICT (content): Merge conflict in config.py
Automatic merge failed; fix conflicts and then commit the result.
```

Opening `config.py` shows the conflict markers Git inserted directly into the file:

CODE

```
DEBUG = True
<<<<<<< HEAD
PORT = 8000
=====
PORT = 5000
>>>>>> feature/login
DATABASE = "prod.db"
```

What Actually Happens

`<<<<<< HEAD` through `=====` is your current branch's version. `=====` through `>>>>>> feature/login` is the incoming branch's version. Git leaves both in the file and waits.

The merge is genuinely paused — `git status` will show the file as unmerged until you resolve it.

Resolving a Conflict

CODE

```
$ git status
Unmerged paths:
  both modified:   config.py

# 1. Edit the file: remove markers, keep the correct line(s)
PORT = 8000

# 2. Stage the resolved file
$ git add config.py

# 3. Finish the merge
$ git commit                # or: git merge --continue
Merge made by the 'recursive' strategy.
```

Staging a previously-conflicted file tells Git "this is resolved." Once every conflicted file is staged, `git commit` (with no message needed — Git pre-fills one) completes the merge commit.

Backing Out

CODE

```
$ git merge --abort          # bail out entirely, back to pre-merge
state
```

`git merge --abort` only works while a conflict is unresolved — it restores your working directory and index to how they were right before the merge started.

COMMON MISTAKE / GOTCHA

- **Mistake:** Forgetting to delete the `<<<<<<<` / `=====` / `>>>>>>>` markers before committing. Git doesn't strip them automatically — if you stage and commit without editing, the literal marker text ships in your file and often breaks the code silently.

QUICK CHECK

- In the conflict markers, which side is HEAD and which is the incoming branch?
- What two steps come after editing out the markers, before the merge is considered done?
- How do you cancel a conflicted merge and return to exactly where you were before it started?

Fast-Forward vs 3-Way Merge

BEGINNER PRIMER

WHAT IT IS

Git merges in one of two ways: sliding a pointer forward with no new commit, or creating a merge commit with two parents.

WHY IT EXISTS

These aren't two commands — they're two outcomes of the same `git merge`, chosen automatically based on whether the receiving branch has moved since the branches diverged.

The Command(s)

CODE

```
$ git switch main
$ git merge feature/login
Updating a1b2c3d..e4f5g6h
Fast-forward
 login.py | 12 ++++++++
 1 file changed, 12 insertions(+)
```

CODE

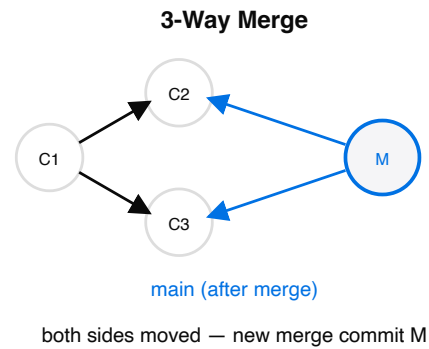
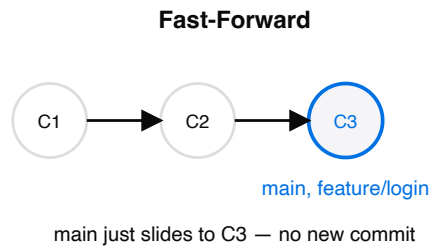
```
$ git merge feature/login
Merge made by the 'recursive' strategy.
 login.py | 12 ++++++++
 1 file changed, 12 insertions(+)
```

Same command, two different results — the output line is the tell: **Fast-forward** vs **Merge made by the '...' strategy**.

What Actually Happens

Fast-forward happens when **main** hasn't gained any commits since **feature/login** branched off — **main**'s tip is a direct ancestor of **feature/login**'s tip. Git just moves the **main** pointer forward to match. No new commit, no merge, just a pointer update.

3-way merge happens when both branches have new commits since they diverged. Git can't just slide the pointer — it has to combine two different histories, so it creates a new merge commit with two parents.



Forcing a Merge Commit

CODE

```
$ git merge --no-ff feature/login
```

Merge made by the 'recursive' strategy.

`--no-ff` forces a real merge commit even when a fast-forward was possible. Teams use this to keep a visible record that a feature branch existed, instead of its commits silently blending into `main`'s straight line.

COMMON MISTAKE / GOTCHA

- **Mistake:** Assuming every merge produces a merge commit you can point to later. A fast-forward merge leaves zero trace that a merge happened — the commits just look like they were made directly on `main`. If your team wants merge history preserved, `--no-ff` is a deliberate policy, not a default.

QUICK CHECK

- What condition on the two branches' histories determines whether a merge is fast-forward or 3-way?
- Why would a team choose `git merge --no-ff` over letting Git fast-forward automatically?

git remote

BEGINNER PRIMER

WHAT IT IS

A remote is a bookmark Git keeps — a short name mapped to the URL of another copy of your repository, usually one sitting on a server like GitHub.

WHY IT EXISTS

Every command you've run so far (`commit` , `branch` , `merge`) only touches the copy of the repo on your machine. To collaborate, you need a way to refer to someone else's copy without retyping its full URL every time. `git remote` gives that copy a short, reusable name.

The Command(s)

CODE

```
$ git remote add origin https://github.com/varunk/my-project.git
# registers a remote named "origin" pointing at that URL
# produces no output on success

$ git remote -v
origin https://github.com/varunk/my-project.git (fetch)
origin https://github.com/varunk/my-project.git (push)

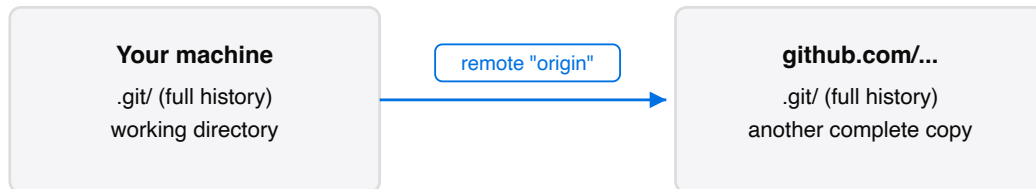
$ git remote remove origin
# deletes the bookmark – does NOT delete anything on the server
```

What Actually Happens

`git remote add <name> <url>` writes an entry into `.git/config`: a name paired with a URL (and, separately, fetch/push refsspecs Git sets up for you). Nothing is downloaded yet — adding a remote just teaches your local repo where another copy lives.

`git remote -v` ("verbose") lists every remote your repo knows about, showing both the fetch URL and push URL for each — they're almost always the same URL, but Git tracks them separately in case you ever want to fetch from one location and push to another.

Local repo vs. remote repo



COMMON MISTAKE / GOTCHA

- **Mistake:** Assuming `origin` is a Git keyword with special behavior. It isn't — it's just the conventional name `git clone` assigns automatically, and everyone keeps using it out of habit. You could name a remote `upstream`, `server`, or `bob` and Git wouldn't care; only the name you typed after `add` matters.

QUICK CHECK

- What two pieces of information does `git remote add` store?
- Does removing a remote with `git remote remove` affect the server copy of the repo?

git push, git pull & git fetch

BEGINNER PRIMER

WHAT IT IS

Three commands that move commits between your local repo and a remote: `push` sends yours up, `fetch` brings theirs down, `pull` brings theirs down and merges them into your work in one step.

WHY IT EXISTS

A local `git remote add` only registers where the other copy lives — it doesn't move any data. These three commands are the actual transfer mechanism, and mixing up `fetch` and `pull` is one of the most common ways beginners get surprised by changes appearing in their working directory.

The Command(s)

CODE

```
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 312 bytes | 312.00 KiB/s, done.
To https://github.com/varunk/my-project.git
  4c3b2a1..a1b2c3d  main -> main
```

CODE

```
$ git fetch origin
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
Unpacking objects: 100% (3/3), 890 bytes | 890.00 KiB/s, done.
From https://github.com/varunk/my-project.git
  a1b2c3d..7f6e5d4  main      -> origin/main
# your working directory and your "main" branch are untouched
```

CODE

```
$ git pull origin main
remote: Enumerating objects: 4, done.
Unpacking objects: 100% (3/3), 890 bytes | 890.00 KiB/s, done.
From https://github.com/varunk/my-project.git
  a1b2c3d..7f6e5d4  main      -> origin/main
```

```
Updating a1b2c3d..7f6e5d4
Fast-forward
 notes.md | 2 ++
 1 file changed, 2 insertions(+)
# fetch + merge happened together; notes.md changed on disk

$ git pull --rebase origin main
# same download, but replays your local commits on top instead of merging
```

What Actually Happens

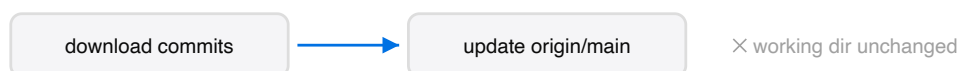
`git push` uploads your local commits to the remote and moves the remote's branch pointer forward to match. It only succeeds if the remote branch is a direct ancestor of what you're pushing — otherwise Git rejects it to avoid overwriting someone else's work.

`git fetch` downloads any new commits and updates a special read-only pointer, `origin/main`, to reflect where the remote's `main` actually is. It never touches your working directory or your local `main` branch — you're just updating your knowledge of what's out there.

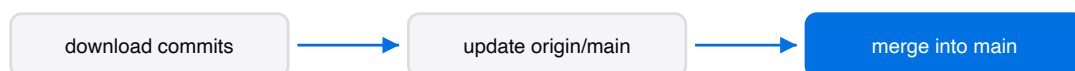
`git pull` is shorthand for `git fetch` immediately followed by `git merge origin/main` (or, with `--rebase`, `git rebase origin/main`). That second step is what actually changes your files and your branch's history.

fetch vs. pull

git fetch



git pull



COMMON MISTAKE / GOTCHA

- **Mistake:** Running `git pull` out of habit when you actually just wanted to check whether the remote has new commits. If you have uncommitted local changes, that automatic merge step can produce a conflict you weren't expecting. `git fetch` followed by inspecting `origin/main` (e.g. `git log main..origin/main`) is the safer way to just look before you merge.

QUICK CHECK

- Which of these three commands can change files in your working directory, and which never can?
- What two commands does `git pull` expand into internally?

git clone

BEGINNER PRIMER

WHAT IT IS

`git clone <url>` downloads a complete copy of a repository — its whole history, all branches, everything — into a new folder on your machine.

WHY IT EXISTS

Joining an existing project means you need a working repo with real history, not an empty one. `git clone` is the one-command way to go from "a URL someone sent me" to a fully functional local repository, already wired up to talk back to where it came from.

The Command(s)

CODE

```
$ git clone https://github.com/varunk/my-project.git
Cloning into 'my-project'...
remote: Enumerating objects: 214, done.
```

```
remote: Counting objects: 100% (214/214), done.
remote: Compressing objects: 100% (140/140), done.
Receiving objects: 100% (214/214), 58.02 KiB | 2.42 MiB/s, done.
Resolving deltas: 100% (86/86), done.

$ cd my-project
$ git remote -v
origin https://github.com/varunk/my-project.git (fetch)
origin https://github.com/varunk/my-project.git (push)

$ git log --oneline -3
7f6e5d4 (HEAD -> main, origin/main) Add notes.md
a1b2c3d Add hero section to landing page
9f8e7d6 Fix typo in footer
```

What Actually Happens

`git clone` does three things in one step: creates a new directory, downloads the entire `.git` history from the URL you gave it, and checks out the default branch into a working directory. It also automatically runs the equivalent of `git remote add origin <url>`, so the clone is immediately ready to `fetch` / `pull` / `push` without any setup.

Compare that to building the same thing by hand: `git init` creates an empty repo with no history and no remote configured. You'd then need `git remote add origin <url>` to register the connection, and `git fetch` plus a checkout to actually pull down the history. `git clone` is those three steps compressed into one.

git clone vs. init + remote add + fetch

`git clone <url>` (one command, does all three below)

`git init`

`git remote add origin`

`git fetch + checkout`

COMMON MISTAKE / GOTCHA

- **Mistake:** Cloning into a folder that already exists and already has content, expecting Git to merge things in. `git clone` wants to create the destination directory itself (or clone into an empty one) — pointed at a non-empty directory, it fails with "destination path already exists and is not an empty directory."

QUICK CHECK

- Which single remote does `git clone` configure automatically, and what is it named?
- What three separate steps does `git clone` replace?

Tracking Branches

BEGINNER PRIMER

WHAT IT IS

A tracking (or "upstream") branch is a local branch remembering which remote branch it corresponds to, so plain `git push` and `git pull` know where to send or fetch data without you naming it every time.

WHY IT EXISTS

Without a tracking relationship, every push or pull needs the remote and branch spelled out explicitly (`git push origin main`). That's tedious and easy to get wrong across many branches. Setting the upstream once means Git remembers, and also lets it warn you when your branch is ahead of or behind the remote.

The Command(s)

CODE

```
$ git push -u origin feature-login
Enumerating objects: 6, done.
Writing objects: 100% (4/4), 512 bytes | 512.00 KiB/s, done.
remote: Create a pull request for 'feature-login' on GitHub by visiting:
remote:   https://github.com/varunk/my-project/pull/new/feature-login
To https://github.com/varunk/my-project.git
 * [new branch]      feature-login -> feature-login
branch 'feature-login' set up to track 'origin/feature-login'.

$ git push
# no arguments needed from now on – Git already knows where "here" goes

$ git branch -vv
main          7f6e5d4 [origin/main] Add notes.md
* feature-login 3d2c1b0 [origin/feature-login] Add login form
old-experiment 9f8e7d6 Try alternate layout
```

What Actually Happens

`-u` (short for `--set-upstream`) pushes the branch and, at the same time, writes a tracking relationship into your local config: "this local branch corresponds to that remote branch." After that, bare `git push` and `git pull` on this branch automatically target `origin/feature-login`.

`git branch -vv` shows the tracking relationship for every local branch in square brackets. `old-experiment` above has no bracket — it has never been pushed, so it isn't tracking anything.

COMMON MISTAKE / GOTCHA

- **Mistake:** Running plain `git push` on a brand-new local branch and getting "fatal: The current branch feature-login has no upstream branch." Git refuses to guess where a never-before-pushed branch should go. The fix is

the one-time `git push -u origin feature-login` — every push after that can drop the arguments.

QUICK CHECK

- What does the `-u` flag on `git push` actually set up, beyond just pushing the commits?
- How can you tell from `git branch -vv` that a branch has never been pushed?

GitHub Basics: Pull Requests & Issues

BEGINNER PRIMER

WHAT IT IS

GitHub is a website that hosts Git repositories and adds collaboration features — like Pull Requests and Issues — on top of plain Git.

WHY IT EXISTS

Git itself has no concept of a "server," a login, or a review process — two people can exchange history over a USB stick and never touch GitHub. GitHub exists to give teams a shared, always-on place to host repos and to layer a review-and-discussion workflow around the commands you already know.

The Command(s)

Everything up to opening the PR is plain Git you've already used — create a branch, commit, push it with an upstream:

CODE

```
$ git switch -c feature-login  
Switched to a new branch 'feature-login'
```

```
$ git add login.html
$ git commit -m "Add login form"
[feature-login 3d2c1b0] Add login form
1 file changed, 24 insertions(+)

$ git push -u origin feature-login
To https://github.com/varunk/my-project.git
 * [new branch]      feature-login -> feature-login
branch 'feature-login' set up to track 'origin/feature-login'.
```

What Actually Happens

A Pull Request (PR) is not a Git object — `git log` and `git show` know nothing about it. It's a GitHub-hosted comparison between two branches, plus a thread for comments, review approvals, and status checks, that ends with someone clicking "Merge."

The typical flow, once your branch is pushed:

1. On GitHub.com, open the repository and click "Compare & pull request" (GitHub shows this banner automatically after a branch push).
2. Choose the base branch (usually `main`) and the compare branch (your feature branch), write a title and description, and click "Create pull request."
3. Teammates review the diff inline, leave comments, and either approve or request changes.
4. Once approved (and any required checks pass), someone clicks "Merge pull request" — this performs the actual merge (or squash, or rebase, depending on team settings) on GitHub's copy of the repo.

An Issue is different from a PR: it's a tracked task, bug report, or discussion item with no code attached to it directly. Issues can be referenced from commit messages or PRs (e.g. writing "Fixes #42") to link the two together, but an Issue itself is just structured text and metadata — title, description, labels, assignee.

COMMON MISTAKE / GOTCHA

- **Mistake:** Treating "commit to GitHub" and "commit to Git" as the same action. Pushing a branch does not open a PR by itself, and merging a PR on GitHub.com updates the remote — your local `main` won't reflect that merge until you run `git pull` or `git fetch` back on your machine.

QUICK CHECK

- Is a Pull Request a Git concept or a GitHub concept? What command would show it in your local history?
- After a PR is merged on GitHub.com, what do you need to run locally to see that merge in your own `main` branch?

Rebase vs Merge

BEGINNER PRIMER

WHAT IT IS

Merge and rebase are two different ways to bring a feature branch's commits into another branch (usually `main`).

WHY IT EXISTS

Both solve the same problem — combining diverged work — but they leave behind very different history. Merge keeps a record of exactly how development really happened; rebase rewrites that record into a straight line, as if the feature had been built on top of the latest main all along.

The Command(s)

CODE

```
$ git checkout main
$ git merge feature
Merge made by the 'ort' strategy.
```

```
search.js | 12 ++++++++
1 file changed, 12 insertions(+)

$ git log --oneline --graph
*   c4d9e21 (HEAD -> main) Merge branch 'feature'
| \
| * a1b2c3d (feature) Add search result highlighting
| * 9f8e7d6 Add search input debounce
* | 7c6b5a4 Update README
|/
* 3d2c1b0 Initial commit
```

CODE

```
$ git checkout feature
$ git rebase main
Successfully rebased and updated refs/heads/feature.

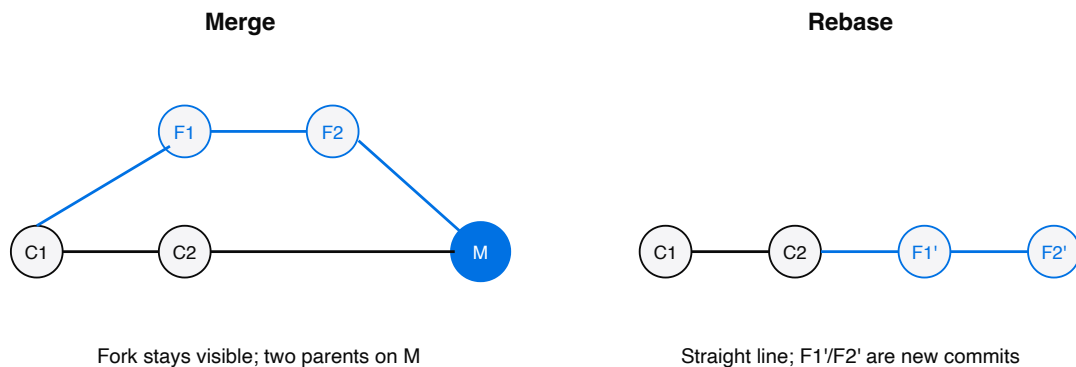
$ git checkout main
$ git merge feature
Updating 7c6b5a4..d4e5f6a
Fast-forward
  search.js | 12 ++++++++
  1 file changed, 12 insertions(+)

$ git log --oneline --graph
* d4e5f6a (HEAD -> main, feature) Add search result highlighting
* e6f7a8b Add search input debounce
* 7c6b5a4 Update README
* 3d2c1b0 Initial commit
```

What Actually Happens

Merge creates a new **merge commit** with two parents — the tip of `main` and the tip of `feature`. Nothing about either branch's existing commits changes; Git just adds one commit on top that ties them together. The fork in history stays visible forever.

Rebase instead takes each commit unique to `feature` and **replays** it, one at a time, on top of the current tip of `main`. Each replayed commit gets the same diff and message as the original, but a new parent — so Git has to give it a brand-new hash. After rebasing, merging `feature` into `main` is just a fast-forward, because `feature`'s history now sits directly after `main`'s.



The trade-off: merge preserves exactly what happened, including when branches diverged, at the cost of a history that can look tangled with many contributors. Rebase produces a clean, linear history that's easy to read — but it rewrites commit hashes, so it must never be run on commits that have already been pushed and pulled by someone else.

COMMON MISTAKE / GOTCHA

- **Mistake:** Rebasing a branch that a teammate has already pulled and built work on top of. Their local history now disagrees with yours — same content, different hashes — and reconciling it means a messy force-push conversation or duplicated commits. The rule of thumb: rebase freely on branches only you use; never rebase a branch other people have fetched.

QUICK CHECK

- Why does a rebased commit get a new hash even though its diff and message are unchanged?

- After a rebase, why does merging the feature branch back into main become a fast-forward?

Interactive Rebase

BEGINNER PRIMER

WHAT IT IS

Interactive rebase lets you edit a stretch of your own commit history — reorder, reword, combine, or delete commits — before anyone else sees them.

WHY IT EXISTS

Real work is messy: "wip", "fix typo", "actually fix typo" commits pile up while you're solving a problem. Interactive rebase turns that messy trail into a small set of clean, well-described commits before you push, without changing the final code.

The Command(s)

CODE

```
$ git log --oneline -3
7c6b5a4 Add password strength meter
9f8e7d6 fix typo in validation message
a1b2c3d Add login form validation

$ git rebase -i HEAD~3
```

Git opens your editor with a todo list, oldest commit first:

CODE

```
pick a1b2c3d Add login form validation
pick 9f8e7d6 fix typo in validation message
pick 7c6b5a4 Add password strength meter
```

```
# Rebase 3d2c1b0..7c6b5a4 onto 3d2c1b0 (3 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup <commit> = like "squash", but discard this commit's log message
# d, drop <commit> = remove commit
#
# These lines can be re-ordered; they are executed from top to bottom.
```

You edit the verbs and save. For example, to fold the typo fix into the first commit and reword the last one:

CODE

```
pick a1b2c3d Add login form validation
fixup 9f8e7d6 fix typo in validation message
reword 7c6b5a4 Add password strength meter
```

CODE

```
[detached HEAD 3f2e1d0] Add login form validation
Date: Sun Jul 19 10:02:11 2026 +0530
2 files changed, 14 insertions(+)
Waiting for your editor to close the file...
[detached HEAD d8c7b6a] Add password strength meter with zxcvbn scoring
Successfully rebased and updated refs/heads/main.
```

What Actually Happens

Git walks the todo list top to bottom, replaying each commit according to its verb.

fixup merges that commit's changes into the one above it and throws away its message; **squash** does the same but lets you edit the combined message. **reword** replays the commit unchanged except it stops to let you rewrite its message.

Every commit from the edit point onward gets a new hash, exactly like a regular rebase — this is the same hash-rewriting mechanic from the previous section, just with more control over what gets replayed.

Verb	Effect
<code>pick</code>	Keep the commit as-is.
<code>reword</code>	Keep the diff, stop to edit the commit message.
<code>edit</code>	Keep the diff, stop before committing so you can amend content.
<code>squash</code>	Merge into the previous commit, combine both messages (editable).
<code>fixup</code>	Merge into the previous commit, discard this commit's message.
<code>drop</code>	Remove the commit entirely.

COMMON MISTAKE / GOTCHA

- **Mistake:** Reordering commits that touch the same lines in a different logical order than they were written. Git replays top to bottom, so if commit B depends on a change introduced in commit A, moving B above A causes a conflict (or worse, silently applies cleanly but leaves broken intermediate states).

QUICK CHECK

- What's the difference between `squash` and `fixup`?
- Why does `git rebase -i HEAD~3` list the oldest of the three commits first, not the newest?

Cherry-Pick

BEGINNER PRIMER

WHAT IT IS

`git cherry-pick` copies one specific commit from anywhere in the repo onto your current branch.

WHY IT EXISTS

Sometimes you need just one fix from another branch, not the whole branch — a hotfix made on a feature branch that also needs to land on `main` right now, without merging unfinished work along with it.

The Command(s)

CODE

```
$ git log --oneline hotfix
e5f6a7b Fix null pointer in payment handler
9f8e7d6 Update payment tests
7c6b5a4 WIP refactor payment flow

$ git checkout main
Switched to branch 'main'

$ git cherry-pick e5f6a7b
[main 4b3a2c1] Fix null pointer in payment handler
Date: Sun Jul 19 09:41:03 2026 +0530
1 file changed, 3 insertions(+), 1 deletion(-)

$ git log --oneline -2
4b3a2c1 (HEAD -> main) Fix null pointer in payment handler
3d2c1b0 Initial commit
```

What Actually Happens

Git takes the diff introduced by `e5f6a7b`, applies it to your current branch, and wraps it in a **brand-new commit** — `4b3a2c1` — with the same message and content but a different parent. The original commit `e5f6a7b` is untouched on `hotfix`; you now have two commits with identical content and different hashes on two different branches.

This is the same "new commit, same content" identity shift you saw with rebase — cherry-pick just does it for a single, hand-picked commit instead of a whole range.

COMMON MISTAKE / GOTCHA

- **Mistake:** Cherry-picking a commit and then later merging the branch it came from. Git doesn't recognize `4b3a2c1` and `e5f6a7b` as "the same change" by hash, so if the diffs don't apply cleanly a second time you'll hit a conflict — or worse, silently get the same fix twice in history with two different hashes.

QUICK CHECK

- After cherry-picking a commit, does the original commit still exist on its source branch?
- Why does the cherry-picked commit get a different hash even though the code change is identical?

git reset vs git revert

BEGINNER PRIMER

WHAT IT IS

`reset` and `revert` both "undo" a commit, but one erases it from history and one adds a new commit that cancels it out.

WHY IT EXISTS

Sometimes you want to pretend a commit never happened (still local, nobody's seen it) — that's `reset`. Other times the commit is already public and other people's work depends on it existing — that's `revert`, which undoes the effect without deleting the record.

The rule to memorize before anything else: `git reset` rewrites history by moving the branch pointer, so it is only safe on commits that are still local and unshared. `git revert` never rewrites anything — it adds a brand-new commit on top — so it's safe to use even on branches other people have already pulled.

The Command(s)

CODE

```
$ git log --oneline
c4d9e21 (HEAD -> main) Add temp debug logging
a1b2c3d Fix payment validation
9f8e7d6 Update tests
3d2c1b0 Initial commit

$ git reset --soft HEAD~1
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   debug.js

$ git log --oneline
a1b2c3d (HEAD -> main) Fix payment validation
9f8e7d6 Update tests
3d2c1b0 Initial commit
```

CODE

```
$ git revert a1b2c3d
[main 7f2e9d0] Revert "Fix payment validation"
 1 file changed, 1 insertion(+), 4 deletions(-)
```

```
$ git log --oneline
7f2e9d0 (HEAD -> main) Revert "Fix payment validation"
a1b2c3d Fix payment validation
9f8e7d6 Update tests
3d2c1b0 Initial commit
```

What Actually Happens

`reset` moves the current branch pointer to a target commit. Depending on the flag, it also rewinds the staging area and/or the working directory to match. `revert` never moves the branch pointer backward at all — it computes the inverse of a commit's diff and commits that inverse as a new, forward-moving commit, so `a1b2c3d` stays in the log exactly as it happened.

Mode	Branch pointer	Staging area	Working directory
<code>--soft</code>	Moves	Unchanged (old commit's changes stay staged)	Unchanged
<code>--mixed</code> (default)	Moves	Reset to match new HEAD (changes become unstaged)	Unchanged
<code>--hard</code>	Moves	Reset to match new HEAD	Reset to match new HEAD — all uncommitted and undone changes are discarded

COMMON MISTAKE / GOTCHA

- **Mistake:** Running `git reset --hard` on a branch that's already been pushed and pulled by teammates, then force-pushing. Their local history now points at commits that no longer exist on the remote — the next

fetch/pull leaves their branch diverged or, if they force-pull, silently drops their own unpushed work. Once a commit is shared, treat it as immutable and use `revert` instead.

Decision rule: already pushed and shared with others? Use `revert`. Still local and never pushed? `reset` is fine.

QUICK CHECK

- Which `reset` mode leaves your changes staged and ready to re-commit?
- Why is `revert` the safer choice on a branch other people have already pulled?

git stash

BEGINNER PRIMER

WHAT IT IS

`git stash` temporarily shelves your uncommitted changes so your working directory is clean again, without committing anything.

WHY IT EXISTS

You're mid-change on one branch when you need to switch to another — a bug report, a different task — but your work isn't ready to commit. Git normally won't let a checkout clobber uncommitted changes; stashing tucks them away so you can switch freely and bring them back later.

The Command(s)

CODE

```
$ git status
On branch main
Changes not staged for commit:
    modified:   search.js

$ git stash
Saved working directory and index state WIP on main: 7c6b5a4 Add search
feature

$ git status
On branch main
nothing to commit, working tree clean
```

CODE

```
$ git checkout hotfix
Switched to branch 'hotfix'
... fix and commit the urgent bug ...

$ git checkout main
Switched to branch 'main'

$ git stash list
stash@{0}: WIP on main: 7c6b5a4 Add search feature

$ git stash pop
On branch main
Changes not staged for commit:
    modified:   search.js

Dropped refs/stash@{0} (3f2e1d094a2b1c7e8f6d5a4b3c2d1e0f9a8b7c6d)
```

What Actually Happens

Stashing takes your tracked, uncommitted changes (staged and unstaged) and saves them as a special commit-like object off to the side, then resets your working directory to match `HEAD`. The stash isn't attached to any branch — it lives independently, so it applies cleanly to whatever branch you're on when you bring it back.

`git stash apply` reapplies the most recent stash but leaves it in the list; `git stash pop` reapplies it and then deletes it from the list. Use `apply` if you might want to apply the same stash to more than one branch.

COMMON MISTAKE / GOTCHA

- **Mistake:** Stashing, forgetting about it, and letting `git stash list` grow to a dozen forgotten entries with no memory of what's in which. Also: by default `git stash` does not include untracked/new files — use `git stash -u` if you need those saved too, otherwise a new file you created will be left sitting in your working directory unstashed.

QUICK CHECK

- What's the difference between `git stash apply` and `git stash pop`?
- Is a stash tied to the branch you created it on? Why does that matter?

git commit --amend

BEGINNER PRIMER

WHAT IT IS

`git commit --amend` replaces your most recent commit with a new one, letting you fix its message or add changes you forgot.

WHY IT EXISTS

You commit, then immediately notice a typo in the message or a file you forgot to `git add`. Making a whole separate "oops" commit for that is noisy; amending folds the fix into the commit that should have had it in the first place.

The Command(s)

CODE

```
$ git commit -m "Add login validaton"
[main 9f8e7d6] Add login validaton
1 file changed, 18 insertions(+)

$ git commit --amend -m "Add login validation"
[main a1b2c3d] Add login validation
Date: Sun Jul 19 11:15:22 2026 +0530
1 file changed, 18 insertions(+)
```

CODE

```
$ git status
On branch main
Untracked files:
  validation.test.js

$ git add validation.test.js
$ git commit --amend --no-edit
[main d4e5f6a] Add login validation
Date: Sun Jul 19 11:15:22 2026 +0530
2 files changed, 26 insertions(+)
```

What Actually Happens

Amend doesn't edit the existing commit object — Git creates a **new** commit combining the previous commit's changes with whatever is currently staged, using the same parent as the original. The branch pointer moves to this new commit, and the old one becomes unreachable (and eventually garbage-collected).

Notice the hash changes each time: `9f8e7d6` becomes `a1b2c3d` becomes `d4e5f6a`. `--no-edit` reuses the previous commit message unchanged; omit it to open your editor and rewrite the message too.

COMMON MISTAKE / GOTCHA

- **Mistake:** Amending a commit that's already been pushed and pulled by someone else. Because amend replaces the commit with a new hash — exactly like rebase and interactive rebase — anyone who already has the old commit now has diverged history and needs a force-push/force-pull to reconcile. Only amend commits that exist solely on your local machine.

QUICK CHECK

- Does `git commit --amend` modify the original commit object, or create a new one?
- What flag lets you amend a commit's staged changes without changing its message?

The Object Model: Blobs, Trees & Commits

BEGINNER PRIMER

WHAT IT IS

Everything in Git — every file, every folder listing, every commit — is stored as one of four object types in a content-addressed database inside

`.git/objects`.

WHY IT EXISTS

Git needs a way to store snapshots cheaply and detect identical content instantly. Splitting "what a file contains" (blob) from "what it's named" (tree) from "when/why/by whom" (commit) lets Git reuse unchanged pieces across every commit instead of copying whole files every time you commit.

The Three Objects You'll Use Daily

CODE

```
$ git cat-file -p a94a8fe5          # a blob: just file bytes
Hello, world!

$ git cat-file -p 4b825dc6          # a tree: names -> hashes
100644 blob a94a8fe5    hello.txt
100644 blob 1f7a7a47    README.md
040000 tree 8f3b2c9a    src

$ git cat-file -p 3b18e512          # a commit: metadata + one tree
tree 4b825dc6
parent 9fceb02c
author Varun Kumar 1721380000 +0530
committer Varun Kumar 1721380000 +0530

Add hello.txt and README
```

What Actually Happens

A **blob** stores raw file content and nothing else — no filename, no permissions, no path. Two files anywhere in your repo with identical content are the exact same blob, referenced from as many trees as needed.

A **tree** is a directory listing: it maps names to object hashes, tagging each entry as a **blob** (a file) or a **tree** (a subdirectory). This is the only place filenames exist in Git's object model.

A **commit** points to exactly one tree (the root of your project at that moment), zero or more parent commits, and metadata — author, committer, timestamp, message. Every commit hash is derived from all of this combined, which is why the same content committed twice with different messages produces different hashes.

commit 3b18e512

```
tree 4b825dc6
parent 9fceb02c
author, message...
```



tree 4b825dc6

```
hello.txt -> a94a8fe5
README.md -> 1f7a7a47
src/ -> 8f3b2c9a
```

blob a94a8fe5

```
"Hello, world!"
```

blob 1f7a7a47

```
# README content
```

tree 8f3b2c9a (src/)

```
main.py -> c3b4d2e1
```

COMMON MISTAKE / GOTCHA

- **Mistake:** Assuming Git "tracks files." It doesn't — it tracks a tree of blobs per commit. Renaming a file doesn't change its blob at all; only the tree entry's name changes, which is why `git mv` and a manual rename+re-add end up identical to Git's rename detection.

QUICK CHECK

- Why don't blobs store filenames?
- If two commits have completely different trees but the same author, message, and timestamp, will they have the same commit hash?

The .git Directory Structure

BEGINNER PRIMER

WHAT IT IS

`.git/` is a plain folder of plain files — no magic, no database server — that holds your entire repository.

WHY IT EXISTS

Every command you've run so far (`commit` , `branch` , `merge` , `stash`) is really just Git reading and writing files under `.git/` . Seeing that layout directly turns "Git magic" into "Git is a filesystem convention."

The Command(s)

CODE

```
$ find .git -maxdepth 1
.git
.git/HEAD
.git/config
.git/description
.git/index
.git/hooks
.git/info
.git/objects
.git/refs
.git/logs
.git/COMMIT_EDITMSG

$ cat .git/HEAD
ref: refs/heads/main

$ find .git/refs -type f
```

```
.git/refs/heads/main
.git/refs/heads/feature/login
.git/refs/remotes/origin/main
```

What Actually Happens

Each entry has one job. Learning them turns every future internals topic into "which file changed."

Path	Purpose
<code>HEAD</code>	A symbolic ref — one line pointing at your current branch (or a raw commit hash if detached).
<code>refs/heads/</code>	One file per local branch, each containing the commit hash it points to.
<code>refs/remotes/</code>	Remote-tracking branches like <code>origin/main</code> — Git's local cache of where the remote's branches last were.
<code>objects/</code>	The blob/tree/commit database from section 032 — every object your repo has ever created.
<code>index</code>	A binary file: the actual on-disk staging area you've been building with <code>git add</code> .
<code>config</code>	Per-repo settings — remotes, user overrides, aliases scoped to this repo only.
<code>logs/</code>	The reflog — a running log of everywhere <code>HEAD</code> and each branch have pointed. Full coverage in section 035.

COMMON MISTAKE / GOTCHA

- **Mistake:** Manually editing files under `.git/` "just to fix something" — e.g. hand-editing `refs/heads/main`. A single typo'd hash can point a branch at a nonexistent or wrong object. Use Git commands (`git update-ref`, `git branch -f`) which validate before writing.

QUICK CHECK

- Which single file tells Git "I am currently on branch main"?
- Is `.git/index` the working directory, the staging area, or the object database?

Refs

BEGINNER PRIMER

WHAT IT IS

A ref is a small text file that holds a commit hash — it's the human-friendly name (like `main`) that points at a specific commit.

WHY IT EXISTS

Commit hashes are 40 hex characters — unusable to type or remember. Refs give Git a layer of stable, readable names that move automatically as you commit, so you say `main` instead of `9fceb02c...` every time.

The Command(s)

CODE

```
$ git rev-parse HEAD
3b18e5123abf9de0c1a2b3c4d5e6f7089a1b2c3d

$ git rev-parse main
3b18e5123abf9de0c1a2b3c4d5e6f7089a1b2c3d

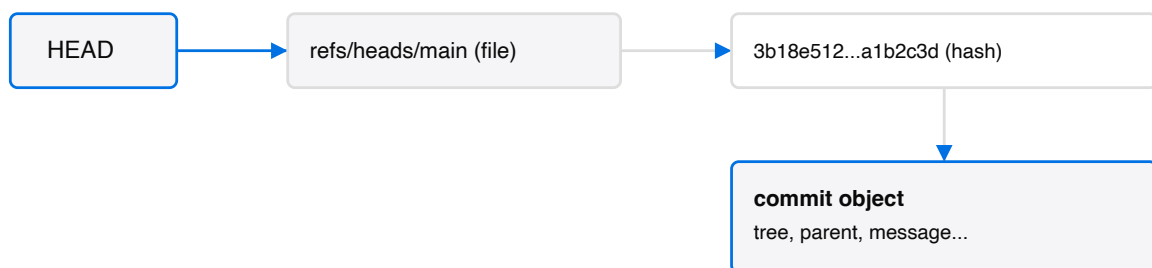
$ cat .git/refs/heads/main
3b18e5123abf9de0c1a2b3c4d5e6f7089a1b2c3d
```

```
$ cat .git/HEAD
ref: refs/heads/main
```

What Actually Happens

A branch like `main` is nothing more than the file `.git/refs/heads/main`, containing one commit hash. When you commit, Git writes the new commit's hash into that same file — that's the entirety of "the branch moved forward."

`HEAD` is a **symbolic ref**: instead of holding a hash directly, it holds a pointer to another ref (`ref: refs/heads/main`). That's the indirection that lets checking out a branch redefine what "current commit" means everywhere in Git.



Tags work the same way but live under `refs/tags/` — a fixed pointer that doesn't move as you commit, unlike a branch. Section 041 covers tags in full.

COMMON MISTAKE / GOTCHA

- **Mistake:** Thinking branches are containers that "hold" commits. A branch holds exactly one hash — the tip. Everything "in" the branch is just commits reachable by walking parent links backward from that one hash.

QUICK CHECK

- What's the difference between a normal ref and a symbolic ref?

- If you run `git rev-parse HEAD` while on branch `main`, how many files does Git actually read to resolve that to a hash?

The Reflog

BEGINNER PRIMER

WHAT IT IS

The reflog is a local, chronological log of every place `HEAD` (and each branch) has pointed on your machine.

WHY IT EXISTS

Operations like `reset --hard`, rebase, and amend (section 031) rewrite history by moving refs — the old commits don't vanish immediately, they just become unreachable from any branch. The reflog remembers where you were before the move, so you can find your way back.

The Command(s)

CODE

```
$ git reflog
3b18e51 (HEAD -> main) HEAD@{0}: commit: fix login validation bug
9fceb02 HEAD@{1}: reset: moving to HEAD~1
c72af90 HEAD@{2}: commit: WIP broken experiment
9fceb02 HEAD@{3}: checkout: moving from feature/login to main
a1e4d88 HEAD@{4}: commit: add login form
7d2c119 HEAD@{5}: commit (amend): initial scaffold
44bb210 HEAD@{6}: commit (initial): initial scaffold
```

What Actually Happens

`git log` shows **ancestry** — it walks parent pointers starting from a ref, so a commit that no branch points to anymore (after a hard reset or rebase) simply disappears from it.

`git reflog` shows **time** — every entry is "HEAD was here, then this happened, now HEAD is there," regardless of whether the old commit is still reachable any other way. That's the fundamental split: log answers "what led to here," reflog answers "everywhere I've been."

`git log` (from main, ancestry only)



`git reflog` (HEAD@{n}, time order, includes orphaned)



c72af90 is invisible to git log (no branch points to it) but reflog still knows it existed.

CODE

```
# recover the "orphaned" WIP commit found above
$ git checkout c72af90
$ git branch recovered-wip
$ git checkout recovered-wip
```

This is the safety net Part 7's recovery techniques rely on — as long as an entry still exists in the reflog, the commit it names hasn't been garbage-collected and can be checked out or reset onto directly.

COMMON MISTAKE / GOTCHA

- **Mistake:** Treating the reflog as permanent or shareable. It's purely local (never pushed, never cloned) and entries expire — unreachable ones are pruned by `git gc` after roughly 30 days by default. It's a recent-history safety net, not an audit log.

QUICK CHECK

- After `git reset --hard HEAD~3`, why do the three "lost" commits still show up in `git reflog` but not in `git log`?
- Does the reflog transfer when you push to a remote or clone the repo elsewhere?

Hashing & Content Addressing

BEGINNER PRIMER

WHAT IT IS

Every object's hash is computed from its own content — so an object's name and its identity are the same thing.

WHY IT EXISTS

Content addressing lets Git detect duplicate content instantly (same bytes always produce the same hash, so it's stored once) and detect corruption instantly (if bytes change, the hash no longer matches, so Git knows something is wrong).

The Command(s)

CODE

```
$ echo -n "hello world" > greet.txt
$ git hash-object greet.txt
95d09f2b10159347eece71399a7e2e907ea3df4

$ echo -n "hello world" > copy.txt    # identical content, different
filename
$ git hash-object copy.txt
95d09f2b10159347eece71399a7e2e907ea3df4    # same hash – same blob
```

What Actually Happens

Git does not hash your raw content alone. It builds a header first — `"blob <byte-length>\0"` — prepends it to the content, and hashes *that combined string* with SHA-1.

CODE

```
# illustrative only – this is conceptually what git hash-object does internally
$ printf 'blob 11\0hello world' | shasum
95d09f2b10159347eece71399a7e2e907ea3df4  -

# "hello world" is 11 bytes, so the header is "blob 11\0"
# git prepends that header, THEN hashes the whole thing
```

The same rule applies to trees (`"tree <len>\0..."`) and commits (`"commit <len>\0..."`) — every object type gets a type-and-length header before hashing.



Two consequences follow directly from this design:

- **Identical content = identical hash.** This is why `git add` on a file you haven't actually changed is a no-op at the object level — Git hashes it, finds the object already exists in `.git/objects`, and writes nothing new.
- **One changed byte = an entirely different hash.** SHA-1 has the avalanche property — no partial similarity. This is why `git commit --amend` (section 026) and every rebased commit (section 031) get brand-new hashes: the content or parent changed, so the hash has to change too, all the way down.

COMMON MISTAKE / GOTCHA

- **Mistake:** Assuming `git hash-object file.txt` gives the same hash as `shasum file.txt`. It won't — `shasum` hashes the raw bytes, `git hash-object` hashes the bytes with Git's `"blob <len>\0"` header prepended. They only match if you replicate that header yourself.

QUICK CHECK

- What two pieces does Git actually feed into SHA-1 when hashing a blob?
- Why does rebasing three commits usually change the hash of all three, even if only the first one's content changed?

git bisect

BEGINNER PRIMER

WHAT IT IS

`git bisect` binary-searches your commit history to find the exact commit that introduced a bug.

WHY IT EXISTS

A bug shows up in production and you know it wasn't there 200 commits ago, but you have no idea which of those 200 commits caused it. Checking out each one by hand is slow and error-prone. Bisect turns "which of 200 commits broke this" into about 8 yes/no tests, since binary search on 200 items takes roughly $\log_2(200)$ steps.

The Command(s)

CODE

```
$ git bisect start
# tell git where the bug currently exists (usually HEAD or a branch tip)
$ git bisect bad
```

```

# tell git the last commit you know was fine
$ git bisect good v2.3.0
Bisecting: 97 revisions left to test after this (roughly 7 steps)
[4f2a9c1e8b3d7a6f9e0c1b2a3d4e5f6a7b8c9d0e] Merge pull request #412 from
feature/cache-layer

# git checks out a commit halfway between good and bad -- test it, then
judge it
$ npm test
...
$ git bisect good          # this commit works fine, bug is later
Bisecting: 48 revisions left to test after this (roughly 6 steps)
[9a8b7c6d5e4f3a2b1c0d9e8f7a6b5c4d3e2f1a0b] Refactor session token
validation

$ git bisect bad          # this commit is broken, bug is earlier or here
Bisecting: 23 revisions left to test after this (roughly 5 steps)
[1c2d3e4f5a6b7c8d9e0f1a2b3c4d5e6f7a8b9c0d] Add retry logic to payment
client

$ git bisect good
...                        # keep repeating good/bad
$ git bisect bad
9a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b is the first bad commit
commit 9a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b
Author: Priya Nair <priya@example.com>
Date:   Tue Jun 3 14:22:07 2026 +0530

    Switch retry backoff to exponential

# done -- return to your original HEAD, discard the bisect state
$ git bisect reset

```

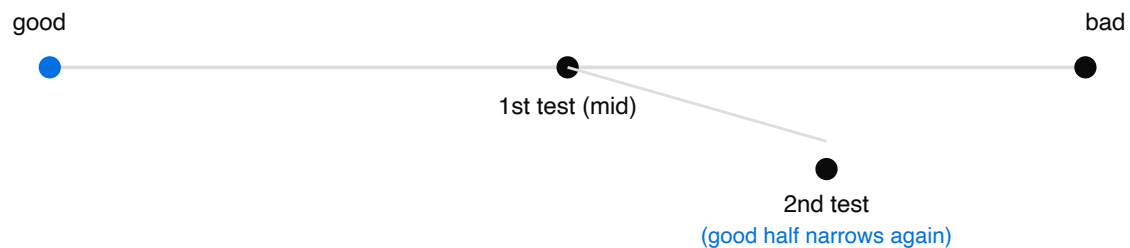
To skip a commit that can't be tested (build broken for unrelated reasons):

CODE

```
$ git bisect skip
```

What Actually Happens

Every time you mark a commit `good` or `bad`, Git narrows the search range by half and checks out a new commit right in the middle for you to test next. You never manually pick which commit to try — Git does that math. This continues until only one commit remains between the last known-good and known-bad state.



Automate the good/bad judgment with a script that exits `0` for good and non-zero for bad:

CODE

```
$ git bisect start
$ git bisect bad HEAD
$ git bisect good v2.3.0
$ git bisect run npm test
running npm test
Bisecting: 48 revisions left to test after this (roughly 6 steps)
...
9a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b is the first bad commit
$ git bisect reset
```

COMMON MISTAKE / GOTCHA

- **Mistake:** Forgetting `git bisect reset` at the end. Bisect leaves you on a detached HEAD pointing at whatever commit was last checked out, not on your original branch — people commit new work there by accident and end up with orphaned commits that don't belong to any branch.

- **Mistake:** Using `git bisect run` with a test command that has side effects (writes to a shared database, sends network requests). Bisect will run it dozens of times against different historical commits, so anything non-idempotent compounds fast.

QUICK CHECK

- Why does bisect need one known-good and one known-bad commit before it starts searching?
- What HEAD state are you in during a bisect session, and why does that matter?
- What must a script passed to `git bisect run` do to signal "this commit is bad"?

Git Hooks

BEGINNER PRIMER

WHAT IT IS

A Git hook is a script that Git runs automatically at a specific point in its workflow, such as right before a commit is created.

WHY IT EXISTS

Teams want to enforce rules — no debug statements, no unresolved merge markers, tests must pass — without relying on every developer remembering to check manually. Hooks let Git itself block or react to an action if the script fails, instead of trusting human discipline.

The Command(s)

Hooks aren't invoked directly — you place an executable script in `.git/hooks/` named after the event, and Git calls it for you. A `pre-commit` hook runs after `git commit` is typed but before the commit object is created:

CODE

```
$ ls .git/hooks/
applypatch-msg.sample    pre-commit.sample
commit-msg.sample        pre-push.sample
post-update.sample       pre-rebase.sample
...

# rename/create the real hook (must be executable, no .sample suffix)
$ mv .git/hooks/pre-commit.sample .git/hooks/pre-commit
$ chmod +x .git/hooks/pre-commit
```

Example `.git/hooks/pre-commit` that rejects a commit containing `console.log` or an unresolved conflict marker:

CODE

```
#!/bin/sh
# .git/hooks/pre-commit

staged_files=$(git diff --cached --name-only --diff-filter=ACM -- '*.js'
'*.ts')

for file in $staged_files; do
    if grep -n "console.log" "$file" > /dev/null; then
        echo "ERROR: console.log found in $file -- remove before committing."
        exit 1
    fi
    if grep -nE '^(<{7}|={7}|>{7})' "$file" > /dev/null; then
        echo "ERROR: unresolved merge conflict marker in $file."
        exit 1
    fi
fi
```

```
done
```

```
exit 0
```

CODE

```
$ git commit -m "add login form"
ERROR: console.log found in src/Login.js -- remove before committing.
# commit is aborted -- nothing was created
```

What Actually Happens

Git calls the hook script and checks its exit code. Exit `0` means "proceed," any non-zero exit code means "abort the operation." For `pre-commit` specifically, an abort means no commit object is ever written — the staging area is untouched, so you just fix the file and re-run `git commit`.

Common hook points: `pre-commit` (before the commit is made), `commit-msg` (validate/rewrite the message), `pre-push` (before objects are pushed to a remote), `post-checkout` (after switching branches).

COMMON MISTAKE / GOTCHA

- **Mistake:** Assuming hooks travel with the repo. Hooks live inside `.git/hooks/`, which is part of the local, untracked Git metadata directory — not the working directory Git tracks with commits. Cloning the repo gives every teammate a fresh, empty `.git/hooks/` with only the `.sample` files, so a hook you wrote is invisible to everyone else unless you distribute it some other way.
- **Mistake:** Forgetting `chmod +x`. A hook script without execute permission is silently skipped by Git — no error, it just never runs.

Tools like **Husky** exist specifically to work around the "hooks aren't shared" problem: they install a hooks config as a tracked file in the repo (e.g. via `package.json`) and wire it up automatically for every teammate on install, so the enforcement rules travel with the codebase instead of living only on one machine.

QUICK CHECK

- Why doesn't a colleague who clones your repo automatically get your `pre-commit` hook?
- What exit code must a hook return to block the Git operation it's attached to?
- Name one problem a tool like Husky solves that raw `.git/hooks/` scripts don't.

Submodules

BEGINNER PRIMER

WHAT IT IS

A submodule embeds one Git repository inside another, pinned to one specific commit.

WHY IT EXISTS

Sometimes a project depends on another project's source (a shared library, a vendored SDK) that has its own history and its own repo. Copy-pasting the code loses the ability to update it cleanly; submodules let the parent repo reference an exact commit of the child repo without merging their histories together.

The Command(s)

CODE

```
$ git submodule add https://github.com/example-org/shared-ui.git  
libs/shared-ui  
Cloning into '/Users/dev/app/libs/shared-ui'...  
remote: Enumerating objects: 842, done.  
Resolving deltas: 100% (512/512), done.
```

```
$ git status
On branch main
Changes to be committed:
  new file:   .gitmodules
  new file:   libs/shared-ui

$ git commit -m "Add shared-ui as submodule"
```

`.gitmodules` records the submodule's URL and path — this file IS tracked and committed:

```
CODE
[submodule "libs/shared-ui"]
    path = libs/shared-ui
    url = https://github.com/example-org/shared-ui.git
```

Cloning a repo that has submodules requires an extra step to actually pull their contents:

```
CODE
$ git clone https://github.com/example-org/app.git
$ cd app
$ ls libs/shared-ui/
# empty -- the submodule directory exists but has no files yet

$ git submodule update --init --recursive
Submodule 'libs/shared-ui' (https://github.com/example-org/shared-ui.git)
registered for path 'libs/shared-ui'
Cloning into '/Users/dev/app/libs/shared-ui'...
Submodule path 'libs/shared-ui': checked out
'7f3a9c1e8b2d4a6f9c0e1b2a3d4e5f6a7b8c9d0e'
```

Or do both in one step at clone time:

```
CODE
$ git clone --recurse-submodules https://github.com/example-org/app.git
```

What Actually Happens

The parent repo doesn't store the submodule's files at all — it stores a special tree entry (a `gitlink`) recording only the exact commit SHA the submodule should be at. The actual submodule code lives in its own `.git` history, checked out into the subdirectory.

Updating the submodule to a newer upstream commit and committing that change in the parent repo is what "bumps the dependency version" — the parent commit's `gitlink` now points at a different SHA.

COMMON MISTAKE / GOTCHA

- **Mistake:** Cloning normally and assuming submodule folders will be populated. `git clone` alone leaves every submodule directory empty — Git only records the `gitlink` pointer, not the content. This is the single most common source of "why is this folder empty" confusion, and the fix is always `git submodule update --init --recursive` (or `--recurse-submodules` at clone time).
- **Mistake:** Committing inside a submodule directory without realizing the parent repo also needs a commit. The submodule's new commit exists in its own history, but the parent still points at the old SHA until you `git add` the submodule path and commit that in the parent repo too.

QUICK CHECK

- What does the parent repo actually store for a submodule — the files, or something else?
- Why can a freshly cloned repo have an empty submodule folder even though `.gitmodules` lists it?
- What two commits are involved when you "update a submodule to the latest version"?

Worktrees

BEGINNER PRIMER

WHAT IT IS

A worktree is a second working directory, checked out from the same repository, that lets you have two branches checked out at once.

WHY IT EXISTS

An urgent hotfix lands while you have a half-finished feature with messy uncommitted changes in your working directory. Stashing and switching branches works, but it's disruptive — especially mid-build or mid-debug session. Worktrees let the urgent branch live in its own directory so your current checkout stays exactly as it is.

The Command(s)

CODE

```
$ git status
On branch feature/checkout-redesign
Changes not staged for commit:
    modified:   src/Checkout.tsx
    modified:   src/api/cart.ts
# messy WIP -- don't want to stash or switch branches right now

$ git worktree add ../hotfix hotfix-branch
Preparing worktree (checking out 'hotfix-branch')
HEAD is now at 3d9e1a2 Fix null pointer in payment webhook

$ cd ../hotfix
$ git status
On branch hotfix-branch
nothing to commit, working tree clean
# separate directory, separate checkout -- original directory untouched
```

List and remove worktrees:

CODE

```
$ git worktree list
/Users/dev/app          3d9e1a2 [feature/checkout-redesign]
/Users/dev/hotfix       3d9e1a2 [hotfix-branch]

$ git worktree remove ../hotfix
```

Create a new branch and its worktree in one step:

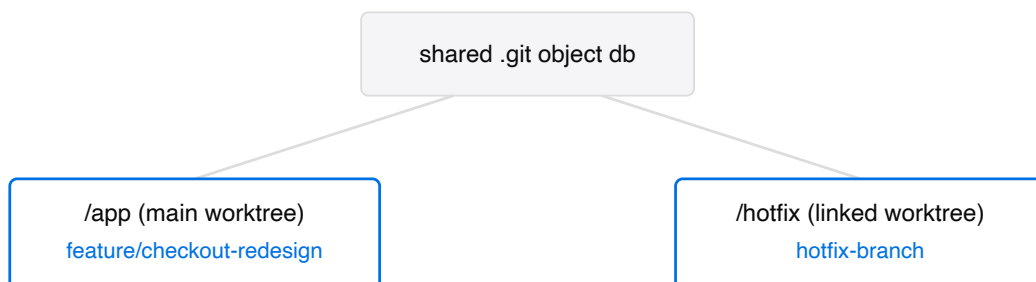
CODE

```
$ git worktree add -b hotfix/payment-webhook ../hotfix origin/main
```

What Actually Happens

Both directories share the same `.git` object database (commits, blobs, trees, config, remotes) — only the working directory files and the currently checked-out branch differ per worktree. This is why it's fast: nothing is re-cloned, only the files for the requested branch are checked out into the new path.

Git enforces that the same branch can't be checked out in two worktrees simultaneously, since that would let you diverge one branch two different ways at once.



COMMON MISTAKE / GOTCHA

- **Mistake:** Trying to check out a branch in a second worktree that's already checked out in the first. Git refuses with "fatal: 'hotfix-branch' is already checked out" — you'd need to finish or remove one checkout first, or use `git worktree add` with a new branch instead.
- **Mistake:** Deleting a worktree directory with `rm -rf` instead of `git worktree remove`. Git still tracks it as a registered worktree afterward and complains on future commands until you run `git worktree prune`.

QUICK CHECK

- What do two worktrees of the same repo share, and what do they keep separate?
- Why can't the same branch be checked out in two worktrees at once?
- How is `git worktree add` different from stashing and switching branches for handling an urgent fix?

Tags

BEGINNER PRIMER

WHAT IT IS

A tag is a named, fixed pointer to one specific commit — most often used to mark a release like `v1.0`.

WHY IT EXISTS

Branches move forward with every commit, which makes them a poor way to mark "this exact commit is what we shipped as version 2.3." Tags never move once created, giving you a stable, memorable name for a specific point in history that survives long after the branch has advanced.

The Command(s)

CODE

```
# lightweight tag -- just a name pointing at a commit
$ git tag v1.0

# annotated tag -- a full object with message, author, date
$ git tag -a v1.1 -m "Release 1.1: fix checkout crash"

$ git tag
v1.0
v1.1

$ git show v1.1
tag v1.1
Tagger: Priya Nair <priya@example.com>
Date:   Tue Jun 3 14:22:07 2026 +0530

Release 1.1: fix checkout crash

commit 7f3a9c1e8b2d4a6f9c0e1b2a3d4e5f6a7b8c9d0e
Author: Priya Nair <priya@example.com>
Date:   Mon Jun 2 09:10:33 2026 +0530

    Fix checkout crash on empty cart
```

Tag an older commit instead of HEAD:

CODE

```
$ git tag -a v1.0.1 3d9e1a2 -m "Hotfix release 1.0.1"
```

Push tags — they are not included in a normal push:

CODE

```
$ git push origin v1.1
$ git push origin --tags    # push all tags at once
```

What Actually Happens

A lightweight tag is nothing more than a ref file under `refs/tags/` pointing straight at a commit SHA — same mechanism as a branch, except it never advances. An annotated tag is a real Git object of its own (a tag object), which points at the commit but also stores a message, tagger name, and date, and can be GPG-signed.

Property	Lightweight tag	Annotated tag
Created with	<code>git tag v1.0</code>	<code>git tag -a v1.0 -m "..."</code>
Own object in the database	No — just a ref pointing at a commit	Yes — a dedicated tag object
Stores message/author/date	No	Yes
Can be GPG-signed	No	Yes (<code>git tag -s</code>)
Typical use	Quick local bookmarks, throwaway markers	Public releases (v1.0, v2.3.1)

COMMON MISTAKE / GOTCHA

- **Mistake:** Running `git push` and assuming your new tag went with it. Tags are never included in a plain `git push` — you must push them explicitly with `git push origin <tagname>` or `git push origin --tags`, or the release marker only exists locally.
- **Mistake:** Using lightweight tags for releases. They carry no author, date, or message, so six months later nobody can tell who cut the release or why — annotated tags are the convention for anything meant to be shared or audited.

QUICK CHECK

- Why doesn't a plain `git push` transfer your local tags to the remote?
- What does an annotated tag store that a lightweight tag doesn't?

- Which tag type creates a new object in the Git object database?

git blame

BEGINNER PRIMER

WHAT IT IS

`git blame` shows, for every line in a file, which commit last changed it and who authored that commit.

WHY IT EXISTS

When you find a confusing or buggy line of code, you usually want context — was this intentional, is there a related ticket, who understands why it's written this way. Blame gets you straight from a line of code to the commit and author responsible, instead of manually scanning history.

The Command(s)

CODE

```
$ git blame src/cart/checkout.py
^3d9e1a2 (Priya Nair    2026-05-11 09:14:03 +0530  1) def
calculate_total(items, coupon=None):
7f3a9c1e (Marcus Ridley 2026-05-14 16:02:41 +0530  2)      subtotal =
sum(i.price * i.qty for i in items)
7f3a9c1e (Marcus Ridley 2026-05-14 16:02:41 +0530  3)      if coupon:
a1b2c3d4 (Priya Nair    2026-06-02 11:47:19 +0530  4)          subtotal =
apply_discount(subtotal, coupon)
^3d9e1a2 (Priya Nair    2026-05-11 09:14:03 +0530  5)      tax = subtotal *
TAX_RATE
9e8f7a6b (Marcus Ridley 2026-06-09 08:30:55 +0530  6)      return
round(subtotal + tax, 2)
```

Limit to a line range, or blame an earlier revision of the file:

CODE

```
$ git blame -L 4,6 src/cart/checkout.py
$ git blame v1.1 -- src/cart/checkout.py
```

What Actually Happens

For each line, Git walks backward through commit history until it finds the most recent commit that changed that specific line, then prints the commit's short hash, author, and timestamp next to it. Lines that haven't changed since the file was created keep pointing at the same original commit (note lines 1 and 5 above share `^3d9e1a2`).

The `^` prefix marks a line traced back to the commit that first created the file in this history (a "boundary" commit). It answers "who last changed this line," not "who wrote the original version" — a line can be reformatted or moved by someone who didn't originally write the logic, and blame will show the reformatter, not the original author.

COMMON MISTAKE / GOTCHA

- **Mistake:** Treating blame as assigning fault for a bug. A line's "last changed by" commit is often a whitespace fix, a rename, or a refactor — not the commit that introduced the actual logic bug. Use `git log -p -- <file>` or `git log -S"<string>"` alongside blame to see the fuller history before pointing fingers.
- **Mistake:** Not knowing about `-w` (ignore whitespace) and `-C` / `-M` (detect moved/copied lines). Without them, a pure reformatting commit shows up as the "author" of every line it touched, burying the real history.

QUICK CHECK

- What exactly does the commit hash next to each line in `git blame` output represent?
- Why might `git blame` point at someone who didn't write the original buggy logic?
- What does the `^` prefix on a blame line indicate?

git clean

BEGINNER PRIMER

WHAT IT IS

`git clean` deletes untracked files from your working directory.

WHY IT EXISTS

Buils, generated files, and stray experiments accumulate in a working directory and clutter `git status`. Clean sweeps them away in one command instead of hunting them down manually with `rm`.

COMMON MISTAKE / GOTCHA

- **Mistake:** Running `git clean -f` without a dry run first. This is **permanent, unrecoverable deletion**. Everything covered in Part 5 — `git reset`, the reflog, recovering "lost" commits — only works because tracked content lives in Git's object database. Untracked files were never committed, never hashed into a blob, never entered the object database at all, so there is no reflog entry, no dangling blob, nothing to recover. Once `git clean -f` deletes an untracked file, it is gone exactly like `rm` deleted it.

The Command(s)

CODE

```
# ALWAYS dry-run first -- shows what WOULD be deleted, deletes nothing
$ git clean -n
Would remove build/bundle.js
Would remove build/bundle.js.map
Would remove notes.txt

# now actually delete untracked files (not directories, not ignored files)
$ git clean -f
```

```
Removing build/bundle.js
Removing build/bundle.js.map
Removing notes.txt
```

Include untracked directories with `-d`:

```
CODE
$ git clean -nd
Would remove build/
Would remove tmp/scratch/
Would remove notes.txt

$ git clean -fd
Removing build/
Removing tmp/scratch/
Removing notes.txt
```

Include files matched by `.gitignore` with `-x` (dangerous — this reaches into `node_modules/`, build caches, etc.):

```
CODE
$ git clean -ndx
Would remove build/
Would remove node_modules/
Would remove .env.local
Would remove notes.txt

$ git clean -fdx
```

What Actually Happens

Clean only ever touches files Git isn't tracking — anything already committed, or currently staged, is left alone entirely. By default it skips both directories and anything matched by `.gitignore`; `-d` and `-x` widen that scope, and combining them (`-fdx`) is the closest thing to "wipe everything not committed."

`git status` and `git clean -n` read the same untracked-file list, so if a file doesn't appear as untracked in status, clean won't touch it either.

QUICK CHECK

- Why can't the reflog help you recover a file deleted by `git clean -f`?
- What does `-d` add to `git clean -f` that plain `-f` doesn't cover?
- Which flag makes `git clean` also remove files matched by `.gitignore`, and why is that risky?

Branching Strategies: Git Flow vs Trunk-Based

BEGINNER PRIMER

WHAT IT IS

A branching strategy is a team's agreed convention for how branches are named, how long they live, and when they merge into the main line of development.

WHY IT EXISTS

Git itself doesn't enforce any workflow — it just lets you branch and merge freely. Without a shared convention, teams collide constantly during merges, release timing gets chaotic, and nobody can tell what's safe to deploy. Git Flow and trunk-based development are the two dominant answers to "what's our convention."

The Command(s)

Neither strategy has one canonical command — they're conventions layered on top of ordinary branch/merge commands. Representative branch names look like this:

CODE

```
# Git Flow style
$ git checkout -b feature/login-page develop
$ git checkout -b release/2.3.0 develop
$ git checkout -b hotfix/2.3.1 main

# Trunk-based style
$ git checkout -b feature/login-page main
# merged back into main within hours or a couple of days, then deleted
$ git branch -d feature/login-page
```

What Actually Happens

Git Flow uses a permanent `develop` branch alongside `main`. Features branch off `develop` and merge back into it; when a release is ready, a `release/*` branch is cut from `develop`, stabilized, then merged into both `main` and `develop`. Urgent fixes get their own `hotfix/*` branches off `main`.

Trunk-based development keeps one long-lived branch (`main` / `trunk`). Feature branches are short — hours to a couple of days — and merge back into `main` frequently, often behind a feature flag if the work isn't ready to ship yet.

	Git Flow	Trunk-Based
Branch lifetime	Long-lived (<code>develop</code> , <code>release/*</code> can live weeks)	Short-lived (hours to a few days)
Release process	Dedicated <code>release/*</code> branch, stabilized separately	Any commit on main can potentially ship; feature flags gate incomplete work
Team size fit	Larger teams, scheduled/versioned releases	Small-to-large teams doing continuous delivery
Merge frequency	Low — features merge once complete	High — multiple times per day per developer
Conflict risk	Higher — branches drift far from each other	Lower — small diffs merged constantly

COMMON MISTAKE / GOTCHA

- **Mistake:** Adopting Git Flow's full branch hierarchy (`develop` , `release/*` , `hotfix/*`) for a small, fast-moving team that ships continuously. The extra long-lived branches add merge overhead and delay without a matching benefit — trunk-based development with feature flags is usually the better fit when there's no fixed release calendar.

QUICK CHECK

- In Git Flow, what branch do feature branches merge into, and what is that branch's role?
- How does trunk-based development ship incomplete features safely without long-lived branches?
- Which strategy has higher merge-conflict risk, and why does branch lifetime cause that?

Recovering Deleted Branches & Commits

BEGINNER PRIMER

WHAT IT IS

When you delete a branch or reset away commits, the commits themselves usually still exist in Git's object database — only the pointer to them is gone.

WHY IT EXISTS

Git almost never deletes commit data immediately just because a branch name or a ref stops pointing at it. This buys you a recovery window, and it exists specifically so that `git branch -D` , `git reset --hard` , and similar commands aren't instantly catastrophic mistakes.

The Command(s)

This is the exact situation you're recovering from — a branch deleted before it was merged:

CODE

```
$ git branch -D feature/payment-retry
Deleted branch feature/payment-retry (was 8f3a1c2).
```

Step 1: Open the reflog. It records every place **HEAD** has pointed, including the tip of the branch you just deleted.

CODE

```
$ git reflog
8f3a1c2 (HEAD -> main) HEAD@{0}: checkout: moving from feature/payment-retry to main
5b9e0af HEAD@{1}: commit: add retry backoff for failed charges
c11d4e0 HEAD@{2}: commit: wire up payment retry queue
a02f7bb HEAD@{3}: checkout: moving from main to feature/payment-retry
```

Step 2: Identify the last commit hash from the deleted branch — here it's **5b9e0af**, the tip before you checked out away from it.

Step 3: Recreate a branch pointing at that hash. This is a normal **git branch** call; the hash just happens to come from the reflog instead of a live ref.

CODE

```
$ git branch recovered-feature 5b9e0af
$ git checkout recovered-feature
Switched to branch 'recovered-feature'

$ git log --oneline -3
5b9e0af (HEAD -> recovered-feature) add retry backoff for failed charges
c11d4e0 wire up payment retry queue
a02f7bb start payment retry feature
```

Step 4 (optional): Rename it back if you want the original name.

CODE

```
$ git branch -m recovered-feature feature/payment-retry
```

What Actually Happens

Deleting a branch removes the ref file (`.git/refs/heads/feature/payment-retry`) — from Part 5 you already know this is just a text file holding a hash. The commit objects it pointed to are untouched in `.git/objects` .

`git reflog` is HEAD's own history of movements, kept separately from branch refs, so it still remembers the hash even after the branch name is gone. Pointing a new branch at that hash is enough — Git walks the commit's parent chain and finds the whole history is still there.

The same technique works for a bad `git reset --hard` : the reflog entry right before the reset shows the commit you reset away from.

COMMON MISTAKE / GOTCHA

- **Mistake:** Waiting too long and assuming the reflog is permanent. By default, unreachable commits are expired from the reflog after 30 days (90 for reachable ones) and can be swept away the next time `git gc` runs. This isn't a hard deadline you need to watch — just don't treat the reflog as infinite, permanent storage. If you know you'll need a commit later, give it a real branch or tag name instead of leaving it dangling.

QUICK CHECK

- Why does deleting a branch not immediately delete its commits?
- If `git branch -D` removed the wrong branch and you've since made several other commits and checkouts, how would you find the right entry in a long reflog?

Undoing a Bad Push

BEGINNER PRIMER

WHAT IT IS

Undoing a bad push means removing a broken commit's effect from a shared branch without disrupting the history everyone else already has.

WHY IT EXISTS

Once a commit is pushed, other people may have already fetched or pulled it. Rewriting shared history out from under them breaks their local repos, so the safe fix adds a new commit that cancels the bad one out, rather than erasing the bad one.

The Command(s)

The scenario: you pushed a commit to `main` that breaks the build, and two teammates have already pulled it.

CODE

```
$ git log --oneline -3
b7c2e91 (HEAD -> main, origin/main) hardcode staging API key in config.js
4f1a0dd fix pagination off-by-one
9e88a3c add rate limit headers
```

Step 1: Revert the bad commit. This creates a new commit that applies the exact inverse of `b7c2e91`.

CODE

```
$ git revert b7c2e91
[main 3d4f5a6] Revert "hardcode staging API key in config.js"
1 file changed, 1 insertion(+), 1 deletion(-)
```

Step 2: Push the revert like any other commit.

CODE

```
$ git push origin main  
To github.com:acme/payments-service.git  
b7c2e91..3d4f5a6  main -> main
```

Everyone else picks up the fix with a normal `git pull` — no coordination required, no rewritten history to reconcile.

What Actually Happens

`git revert` doesn't touch the bad commit at all — `b7c2e91` is still right there in the log, forever. It computes the diff that commit introduced and commits the opposite diff on top.

History only ever grows forward. Anyone who already has `b7c2e91` in their local `main` just fast-forwards to include the new revert commit, the same as pulling any other change.

This is why revert is the default answer for anything already pushed to a branch other people pull from.

COMMON MISTAKE / GOTCHA

- **Mistake:** Reaching for `git reset --hard HEAD~1` followed by a force-push, because it feels like a "cleaner" fix that removes the bad commit entirely. On a shared branch this rewrites history that others have already pulled — see [Force-Push Dangers](#) for exactly what breaks. Amending and force-pushing a correction is only reasonable if you are certain absolutely no one has fetched the bad commit yet, e.g. you pushed it thirty seconds ago and can confirm no one has pulled.

QUICK CHECK

- Why does `git revert` stay safe for commits others have already pulled, when a hard reset does not?

- After reverting a bad commit, does the original bad commit disappear from `git log`? Why or why not?

Force-Push Dangers

BEGINNER PRIMER

WHAT IT IS

`git push --force` overwrites a remote branch to match your local branch exactly, no matter what was on the remote before.

WHY IT EXISTS

After rewriting local history — an interactive rebase, an amend, a reset — your branch and the remote's branch have diverged, and a normal push is rejected. Force-push exists to let you deliberately replace the remote's history with your own when that divergence is intentional.

The Command(s)

What a plain force-push looks like:

CODE

```
$ git push --force origin feature/checkout-refactor
+ a1b2c3d...e4f5a6b feature/checkout-refactor -> feature/checkout-refactor
(forced update)
```

The safer version — `--force-with-lease` — checks that the remote branch still has the commit you last saw before overwriting it:

CODE

```
$ git fetch origin
$ git push --force-with-lease origin feature/checkout-refactor
+ a1b2c3d...e4f5a6b feature/checkout-refactor -> feature/checkout-refactor
```

(forced update)

If a teammate pushed to that branch after your last fetch, `--force-with-lease` refuses instead of overwriting their work:

CODE

```
$ git push --force-with-lease origin feature/checkout-refactor
To github.com:acme/payments-service.git
 ! [rejected]          feature/checkout-refactor -> feature/checkout-refactor
(stale info)
error: failed to push some refs
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository
pushing
hint: to the same ref.
```

At that point, fetch, look at what changed, and decide how to reconcile before pushing again — don't just force through it.

What Actually Happens

A normal `git push` only succeeds if it's a fast-forward — your local branch must contain everything the remote branch has. Force-push skips that check and simply sets the remote ref to your local commit hash.

Any commit that was on the remote but isn't in your local history — including one a teammate pushed five minutes ago that you haven't fetched yet — gets orphaned. It still exists as an object until garbage collected, but nothing on the remote points to it anymore, and your teammate's next pull will silently diverge or conflict.

`--force-with-lease` fixes this by refusing to push unless the remote's current tip still matches what your local `refs/remotes/origin/...` last recorded — effectively "force-push, but only if nobody moved the remote since I last looked."

When It's Fine vs. When It's Dangerous

Fine	Dangerous

Your own feature branch, not shared with anyone	Any branch other people pull from — <code>main</code> , <code>develop</code> , release branches
After an interactive rebase or amend, before anyone has pulled it	A branch a teammate has already fetched or based work on
Cleaning up your own PR branch before review	"Fixing" a shared branch under time pressure without checking who else pushed

COMMON MISTAKE / GOTCHA

- **Mistake:** Using plain `--force` out of habit even on branches other people work on, because it "usually works fine." It only feels safe because most of the time nobody happens to push in that exact window. `--force-with-lease` costs nothing extra and turns the dangerous case into a rejected push instead of a silently lost commit — there's rarely a good reason to prefer plain `--force` on anything but a solo branch.

QUICK CHECK

- What exactly does `--force-with-lease` check before it allows the push to proceed?
- You amended your last commit on a feature branch that only you work on. Is force-pushing it safe? What would change your answer?

Command Cheat Sheet

Basics

Situation	Command
See what's changed, staged, or untracked	<code>git status</code>

Stage a specific file	<code>git add <file></code>
Stage everything changed	<code>git add .</code>
Stage only part of a file's changes	<code>git add -p</code>
Remove a tracked file (working dir + staging)	<code>git rm <file></code>
Stop tracking a file without deleting it	<code>git rm --cached <file></code>
Rename or move a tracked file	<code>git mv <old> <new></code>
Save a snapshot of staged changes	<code>git commit -m "message"</code>
Stage all tracked changes and commit in one step	<code>git commit -am "message"</code>
Fix the last commit's message or contents	<code>git commit --amend</code>
Permanently exclude a pattern of files from tracking	add pattern to <code>.gitignore</code>
View commit history	<code>git log --oneline --graph --all</code>
Inspect one commit's metadata and diff	<code>git show <hash></code>

Branching & Merging

Situation	Command
List / create / delete a branch	<code>git branch</code> / <code>git branch <name></code> / <code>git branch -d <name></code>
Switch to an existing branch (modern)	<code>git switch <branch></code>
Switch to an existing branch (legacy, still common)	<code>git checkout <branch></code>
Create and switch to a new branch in one step	<code>git switch -c <branch></code>
Discard working-directory changes to a file	<code>git restore <file></code>

Unstage a file, keeping the edit	<code>git restore --staged <file></code>
Combine another branch into the current one	<code>git merge <branch></code>
Merge but always keep a merge commit (skip fast-forward)	<code>git merge --no-ff <branch></code>
Bail out of a merge with conflicts	<code>git merge --abort</code>

Remote Collaboration

Situation	Command
Add / list a remote	<code>git remote add origin <url></code> / <code>git remote -v</code>
Send local commits to the remote	<code>git push</code>
Push a new branch and set tracking, first time	<code>git push -u origin <branch></code>
Download and merge remote changes	<code>git pull</code>
Download remote changes without merging	<code>git fetch</code>
Copy a remote repo locally, with full history	<code>git clone <url></code>
See tracking relationships for all branches	<code>git branch -vv</code>

Rewriting History

Situation	Command
Replay current branch's commits onto a new base (linear history)	<code>git rebase <branch></code>

Edit, squash, reorder, or drop recent commits	<code>git rebase -i HEAD~<n></code>
Copy one specific commit onto the current branch	<code>git cherry-pick <hash></code>
Undo local commits, keep changes staged	<code>git reset --soft <hash></code>
Undo local commits, unstage changes (default mode)	<code>git reset --mixed <hash></code>
Undo local commits, discard changes entirely	<code>git reset --hard <hash></code>
Undo a commit that's already pushed/shared	<code>git revert <hash></code>
Shelve uncommitted work to switch branches	<code>git stash</code>
Reapply the most recent stash	<code>git stash pop</code>

Advanced & Recovery

Situation	Command
Mark a release point (simple)	<code>git tag v1.0</code>
Mark a release point (with metadata)	<code>git tag -a v1.0 -m "message"</code>
Find the commit that introduced a bug	<code>git bisect start</code> , then <code>git bisect bad</code> / <code>git bisect good <hash></code>
See who last changed each line of a file	<code>git blame <file></code>
Preview which untracked files would be deleted	<code>git clean -n</code>
Delete untracked files for real	<code>git clean -f</code>
Add another repo as a dependency pinned to one commit	<code>git submodule add <url> <path></code>
Check out a second branch into its own folder	<code>git worktree add ../<path> <branch></code>

Recover a commit that
reset/rebase seems to have lost

`git reflog`, then `git branch <name> <hash>`